Sisteme și algoritmi distribuiți
Curs 2

Cuprins

- Modele de comunicație
- Comunicație prin mesaje
- Mesaje persistente
- Ceasuri

Comunicație

În limbajul natural (uman) exprimarea unui mesaj cuprinde:

- Dialect
- Limbă
- Cuvînt
- Alfabet

Comunicație

Premise necesare pentru comunicație:

- Sursa și destinația sunt conectate fizic
- Sursa alege alfabetul, limba etc., în care codifică mesajul
- Destinația decodifică mesajul (recunoaște alegerile sursei)

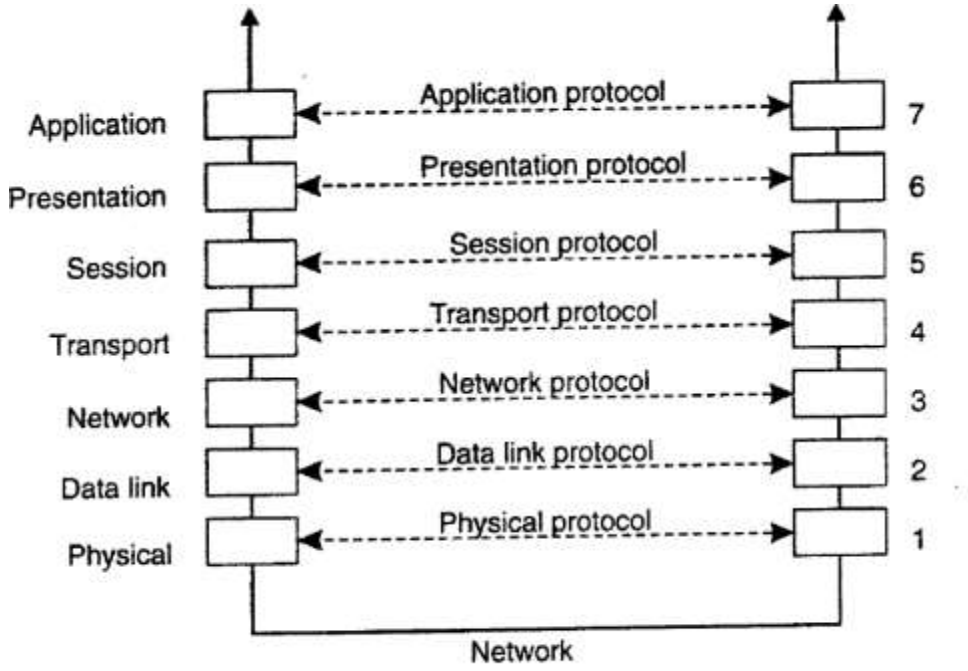
Modele de comunicație

A comunică cu B:

- A produce un mesaj
- Apelează o primitivă pentru a trimite mesajul
- B apelează o primitivă pentru a primi mesajul

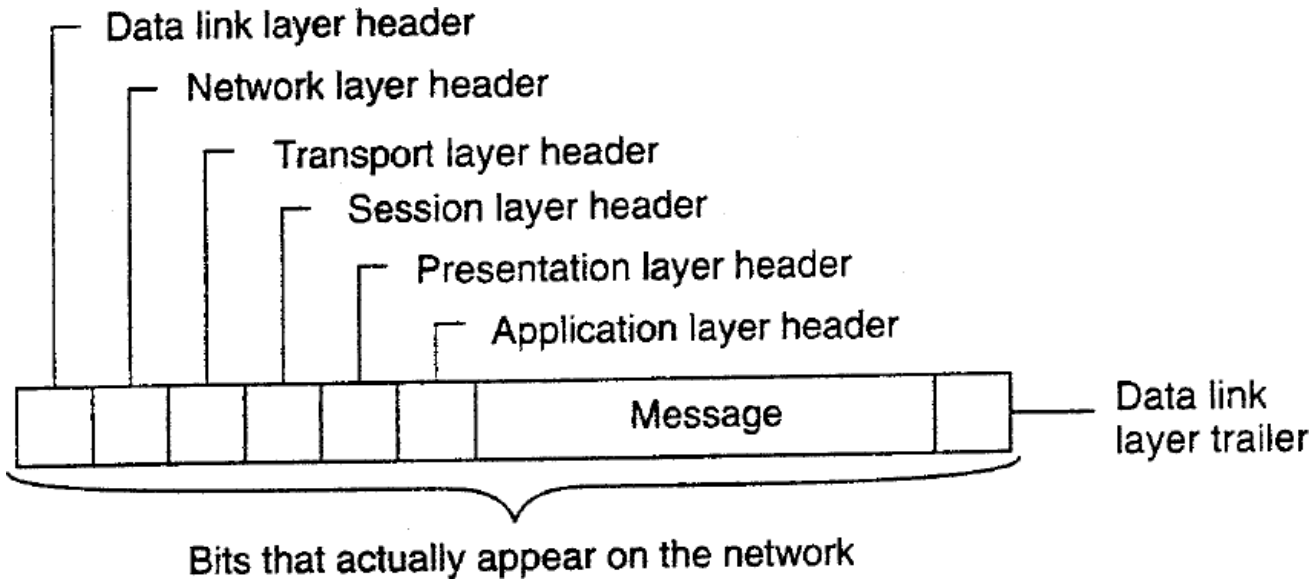
Protocoale de comunicație

Protocol = algoritm care specifică formatul, conținutul și sensul mesajelor trimise și primite de către un proces.



Protocoale de comunicație

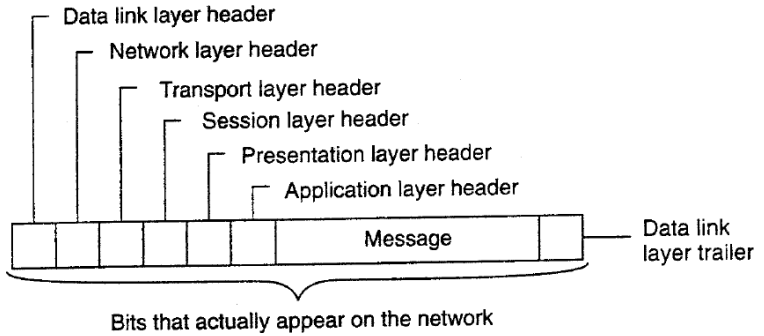
Protocol = algoritm care specifică formatul, conținutul și sensul mesajelor trimise și primite de către un proces.



Ex.: User Datagram Protocol (UDP)

Protocol de comunicație la nivelul (layer) de transport

- Fără conexiune = sursa comunică pachete către destinație fără a negocia o conexiune (implicit fără gestiunea pachetelor)
- Lightweight = nu necesită ordonare sau istoric de pachete; comunicația mesajului începe de la primul pachet
- Singurul mecanism de siguranță este *checksum*, nu există verificarea succesului transmisiei.
- Folosit în aplicațiile de streaming, broadcasting



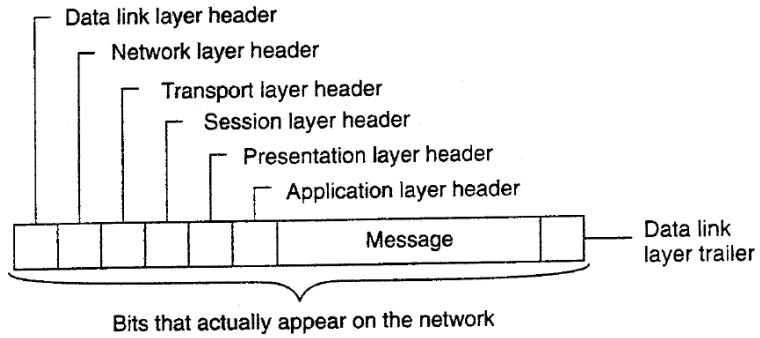
Ex.: User Datagram Protocol (UDP)

Offset	Octet	0								1								2								3							
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	Source Port																Destination Port															
4	32	Length																Checksum															
8	64	Data																															
12	96																																
⋮	⋮																																

Ex.: Transmission Control Protocol (TCP)

Protocol de comunicație la nivelul (layer) de transport

- Orientat pe conexiune = înainte de interschimbarea de mesaje dintre sursă și destinație, se realizează o conexiune.
- Heavyweight = primele 3 pachete necesare pentru conexiune prin socket.
- Mecanisme de ACK, retransmisie și time-out
- Cuprinde trei etape:
  - Stabilire conexiune
  - Transfer date
  - Oprire conexiune
- Divide mesajul în porțiuni și creează segmente de dimensiuni fixe

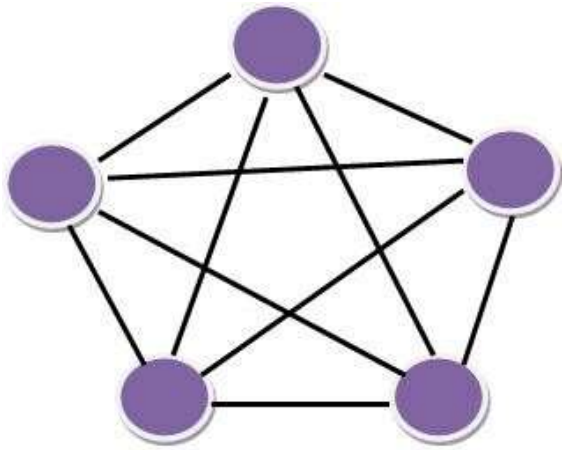


Ex.: Transmission Control Protocol (TCP)

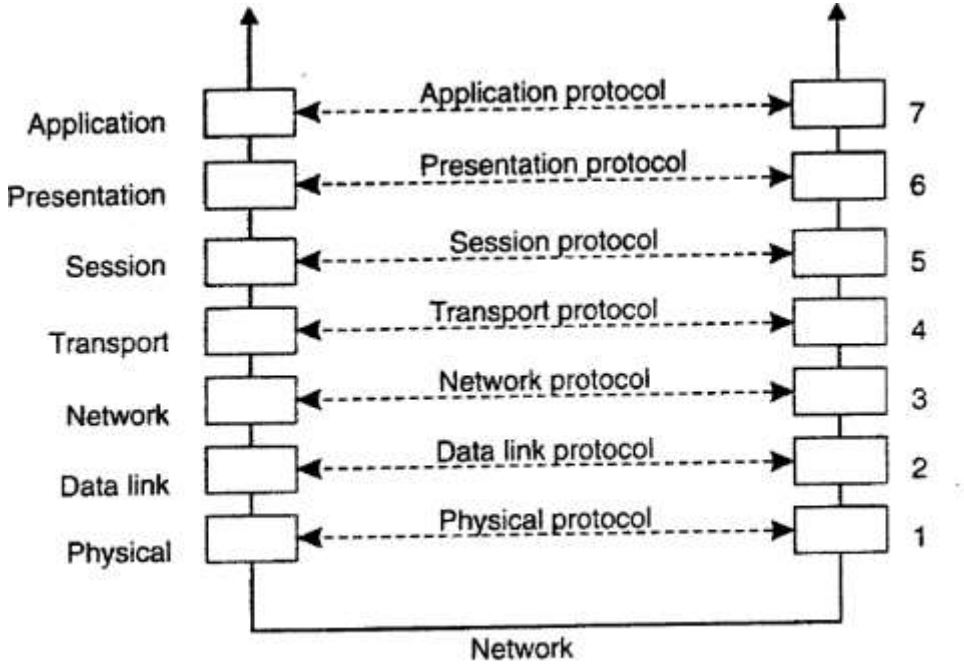
Offset	Octet	0								1								2								3							
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	Source Port																Destination Port															
4	32	Sequence Number																															
8	64	Acknowledgement Number (meaningful when ACK bit set)																															
12	96	Data Offset				Reserved				CWR	ECE	URG	ACK	PSH	RST	SYN	FIN	Window															
16	128	Checksum																Urgent Pointer (meaningful when URG bit set)[18]															
20	160	[Options] If present, Data Offset will be greater than 5. Padded with zeroes to a multiple of 32 bits, since Data Offset counts words of 4 octets.																															
⋮	⋮																																
56	448																																
60	480	Data																															
64	512																																
⋮	⋮																																

Modele de comunicație

1. *Comunicație prin mesaje*
2. Comunicație prin mesaje persistente
3. Comunicație colectivă

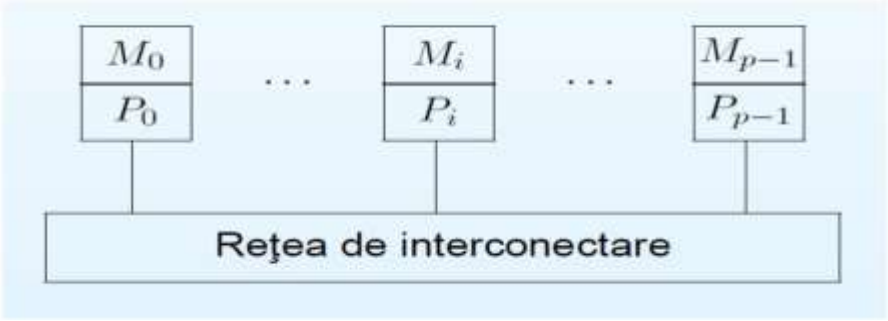


http://4.bp.blogspot.com/-LLvGwl5z-_4/UMadzzwP7tI/AAAAAAAAAHY/2AxHYokPHzk/s1600/all+channel.jpg



Comunicația prin mesaje: Modelul SPMD

- MIMD cu memorie distribuita
- Paradigma **SPMD** (Single Program Multiple Data): toate procesoarele execută același program, dar fiecare utilizează un set propriu de date.
- În general, execuția programului nu este sincronă;
- Deși toate procesoarele văd același program, instrucțiunile executate nu sunt identice



Modelul SPMD

- Procesoarele sunt numerotate $0, \ldots, p$
- Numerotarea nu este statică, ci se realizează la momentul execuției.
- Există primitive care returnează adresa unui procesor (e.g. MPI_rank)
- Procesoarele pot executa instrucțiuni diferite în funcție de adresa lor

1. rank ← adresa proprie
2. **dacă** rank = 0 **atunci**
    1. a ← 1
3. **altfel**
    1. a ← 0

Modelul SPMD - variabile

- O variabilă a unui program SPMD este multiplicată (de p ori) : fiecare procesor deține un exemplar, asupra căruia are control complet.
- Un procesor nu poate modifica variabile din memoria altui procesor.
- Putem interpreta variabila a ca un vector cu p elemente: fiecare procesor P_i deține componenta i a vectorului. Cu toate acestea i reprezintă un index global al datelor din a.
- Programul inițializează a cu 0 pe toate componentele, cu excepția primei componente (care este 1).

Modelul SPMD - problemă

Inițializați un vector a de dimensiune n cu zero, mai puțin elementul a_m, care să fie 1.

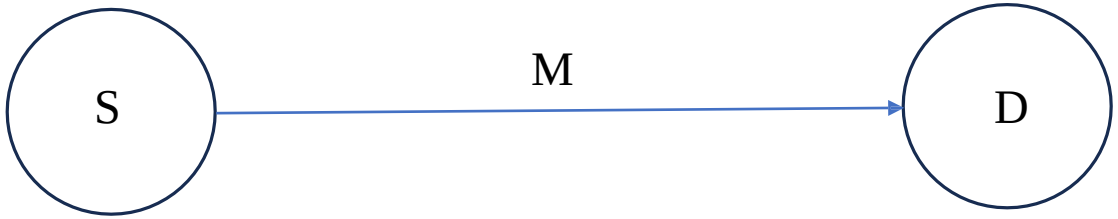
- Cum distribuim vectorul a celor p procesoare?

- Cum facem efectiv inițializarea?

Modelul de comunicație prin mesaje

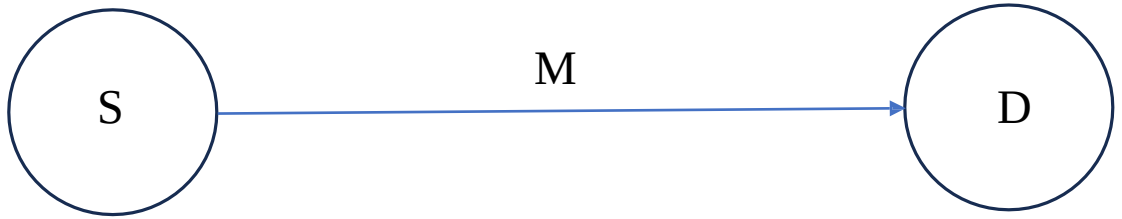
- Singura modalitate de comunicare între procesoare este transmiterea de mesaje

- **Operația de bază**:



Modelul de comunicație prin mesaje

- **Operația de bază**:

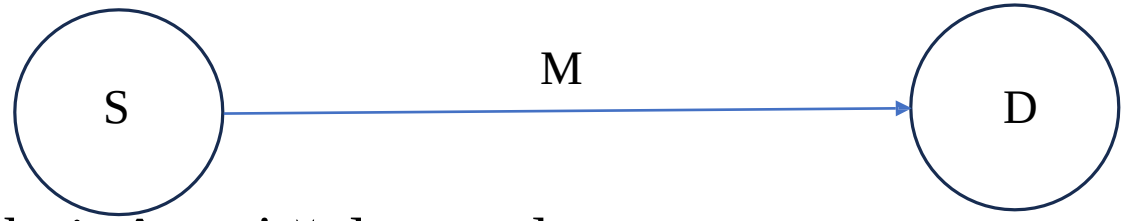


- Procesorul sursă transmite prin rutina **send**
- Procesorul destinație recepționează prin rutina **recv**

- Sintaxă generală:
  - send(date, dest)
  - recv(date, sursa)
- date = locație (buffer) din care se preiau/depun mesajele transmise
- sursa/dest = adresa procesorului cu care se comunică

MP - corectitudine

- **Operația de bază**:



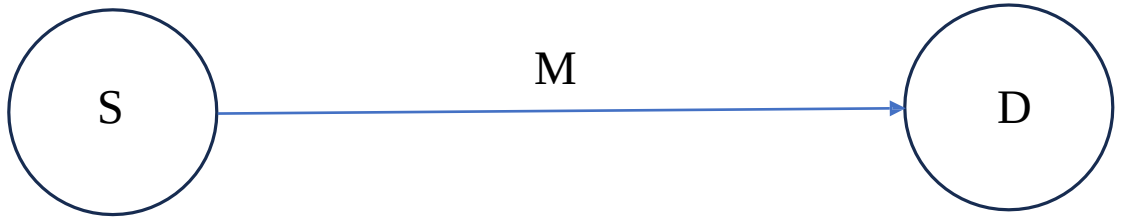
- Orice operație de **send** trebuie însoțită de una de **recv**
- Per ansamblu, vom avea perechi **send-recv**

Exemplu: Procesorul i transmite un mesaj M vecinilor de la stânga, respectiv dreapta (pe o topologie inel)

1. **dacă** rank = i **atunci**
    1. send(M, $(i+1) mod\ p$);
    2. send(M, $(i-1) mod\ p$);
2. **Altfel dacă** rank = $(i+1) mod\ p$ **atunci** recv(M, i);
3. **Altfel dacă** rank = $(i-1) mod\ p$ **atunci** recv(M, i);

MP - sincronizare

- **Operația de bază**:

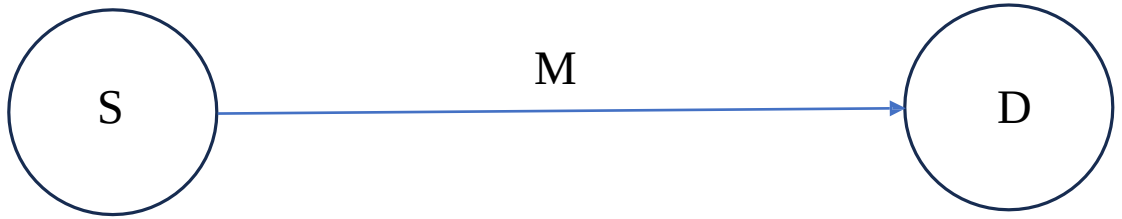


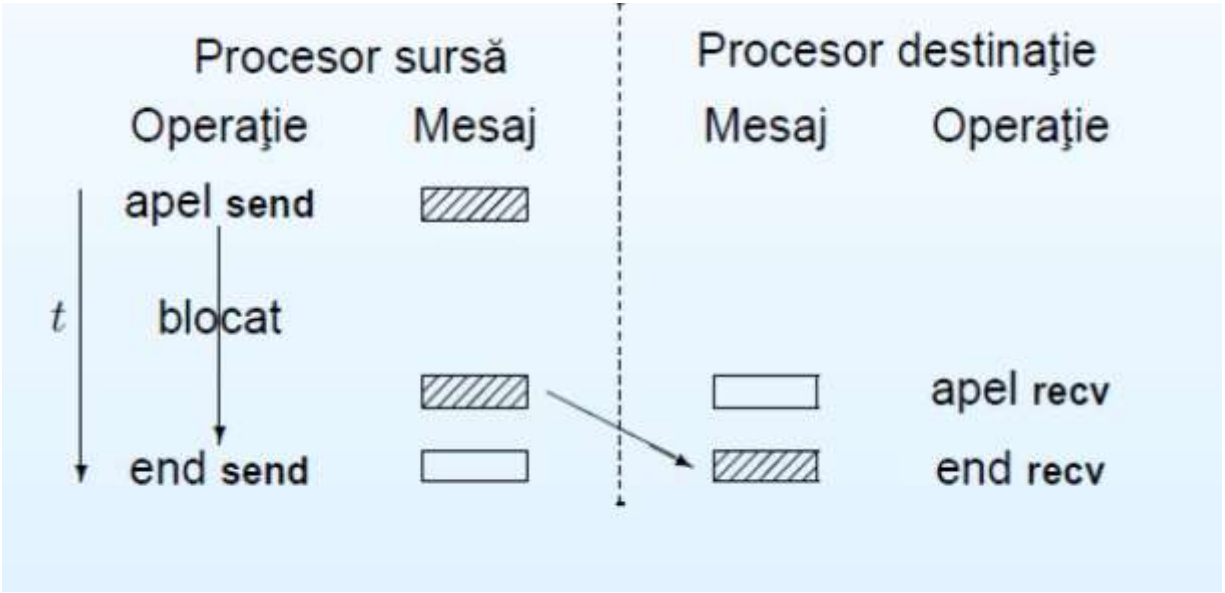
- Momentul apelului primitivei **send** este în general diferit față de momentului apelului **recv**

- Ce se întâmplă din momentul apelului primei primitive până la finalizarea comunicației?
  - Răspunsuri posibile: (i) așteaptă (**comunicație blocantă**); (ii) poate executa alte operații (**comunicație non-blocantă**)

MP – comunicație blocantă

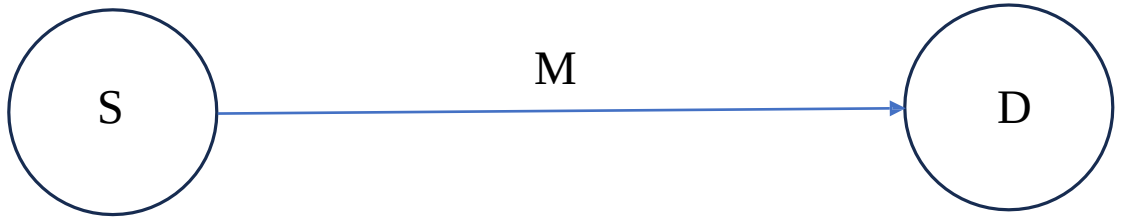
- **Operația de bază**:

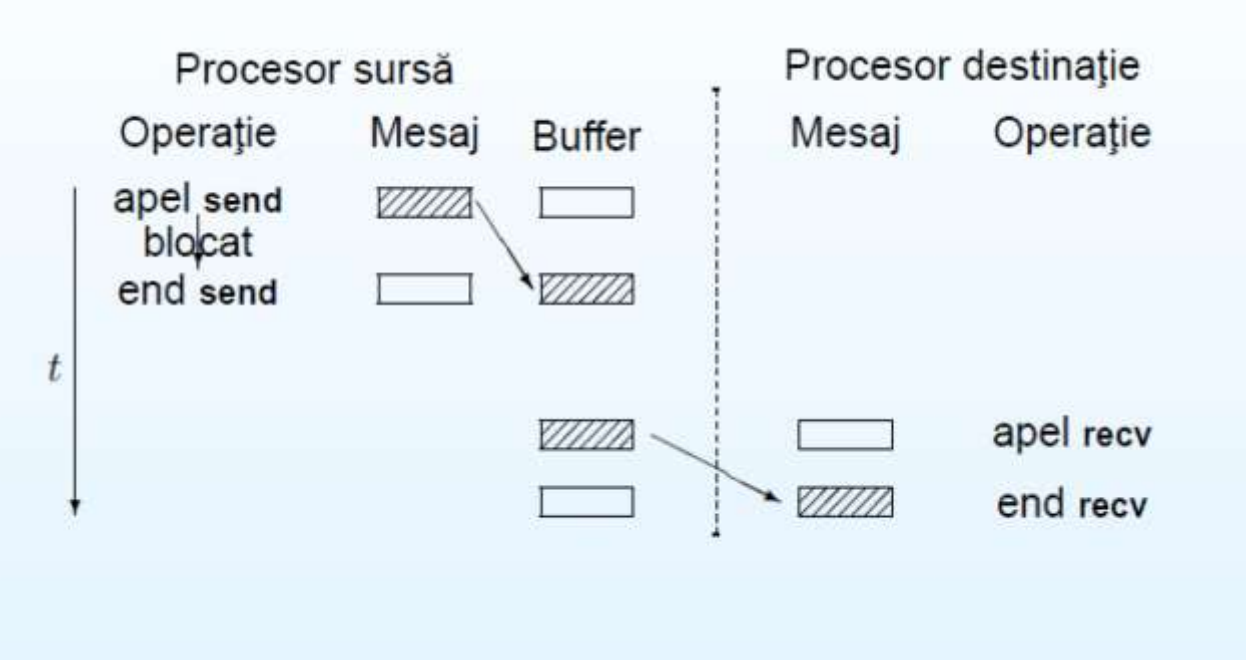




MP – comunicație blocantă prin buffer

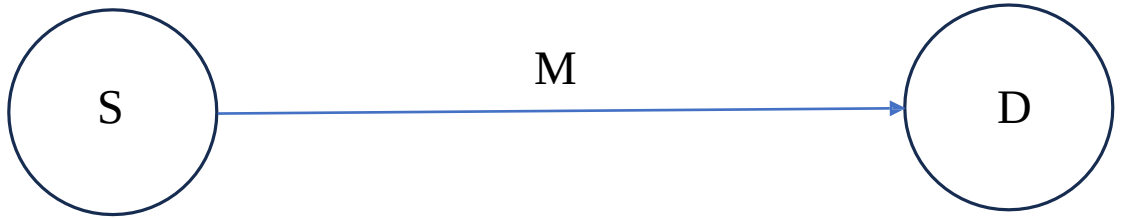
- **Operația de bază**:

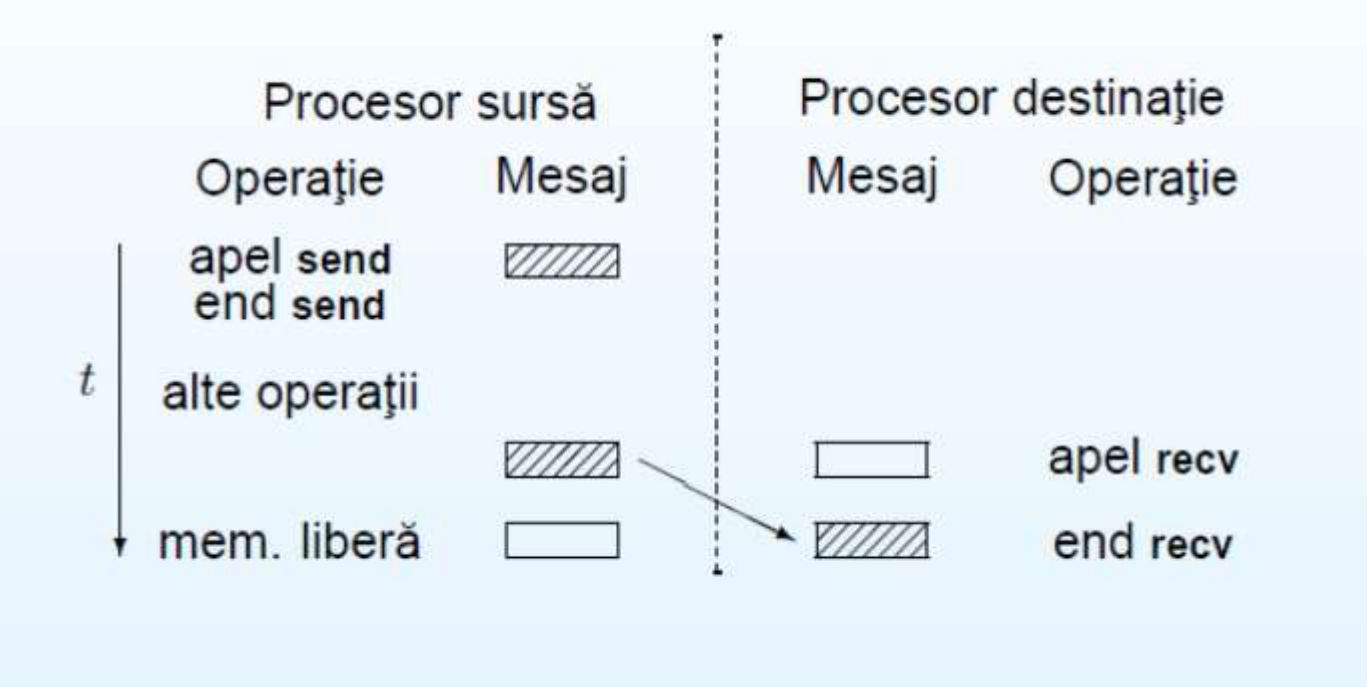




MP – comunicație non-blocantă

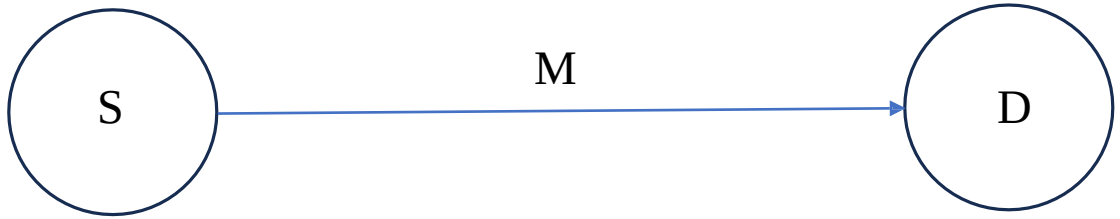
- **Operația de bază**:





MP – comunicație non-blocantă

- **Operația de bază**:



Alte primitive:

- **Așteaptă (Wait) –** așteptarea terminării comunicației

Utilizare:

1. **send**(*date*, dest);
2. execută operații care nu modifică date;
3. **așteaptă** terminarea **send**;
4. modifică date;

MP – comparație

- Comunicația non-blocantă asigură o ocupare mai bună a procesoarelor

- Erorile de programare:
  - Comm blocantă: blocarea execuției
    - send(m1, (rank + 1) mod p)
    - recv(m2, (rank - 1) mod p)

  - Comm non-blocantă: alterarea zonei de memorie după apelul **send**

Standardul MPI (Message-Passing Interface)

- Standard care descrie primitivele de comunicație în paradigma SPMD
- Implementări: OpenMPI, mpich2 etc.
- O rutină MPI se execută în cadrul unui grup de procesoare (comunicator)
- Într-un comunicator procesoarele sunt numerotate $\{0, \ldots, p-1\}$

C:
int MPI_Comm_rank(MPI_Comm com, int * my_rank)
int MPI_Comm_size(MPI_Comm com, int *p)

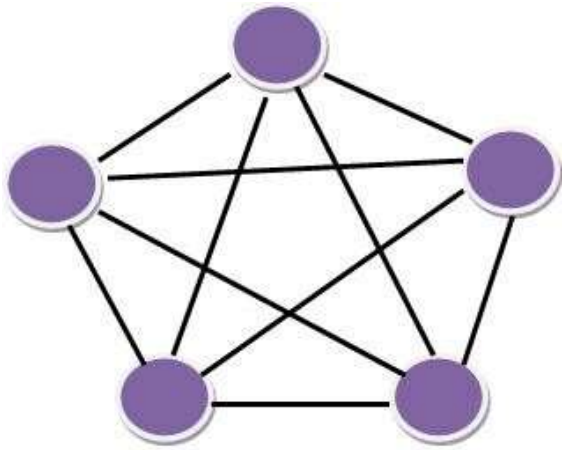
Python:
int comm.Get_rank()
int comm.Get_size()

MPI primitives

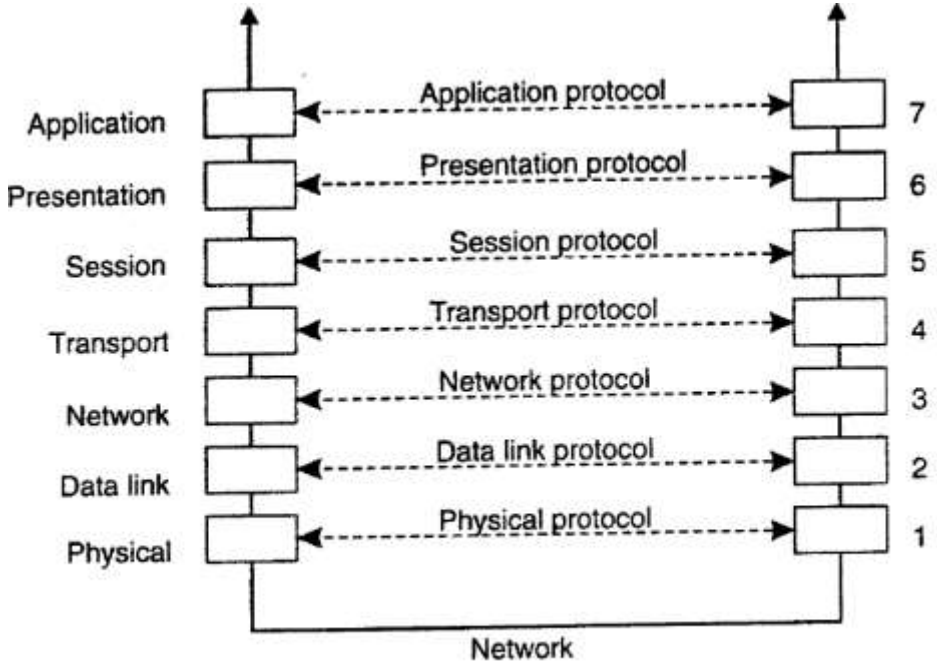
Primitive	Meaning
MPI_bsend	Append outgoing message to a local send buffer
MPI_send	Send a message and wait until copied to local or remote buffer
MPI_ssend	Send a message and wait until receipt starts
MPI_sendrecv	Send a message and wait for reply
MPI_isend	Pass reference to outgoing message, and continue
MPI_issend	Pass reference to outgoing message, and wait until receipt starts
MPI_recv	Receive a message; block if there is none
MPI_irecv	Check if there is an incoming message, but do not block

Modele de comunicație

1. Comunicație prin mesaje
2. *Comunicație prin mesaje persistente*
3. Comunicație colectivă



http://4.bp.blogspot.com/-LLvGwl5z-_4/UMadzzwP7tI/AAAAAAAAAHY/2AxHYokPHzk/s1600/all+channel.jpg



Comunicație prin mesaje persistente (MQS)

Message Queuing Systems = sistem în care aplicațiile comunică prin inserarea de mesaje în cozi specifice; "O abstractizare a căsuței poștale"

- Sursa (producer) are garanția că mesajul său va fi eventual inserat în coada destinatarului (receiver)
- Nu avem garanții despre momentul când (sau dacă) mesajul va fi citit
- Sursa și destinatarul execută complet independent unul de celălalt.



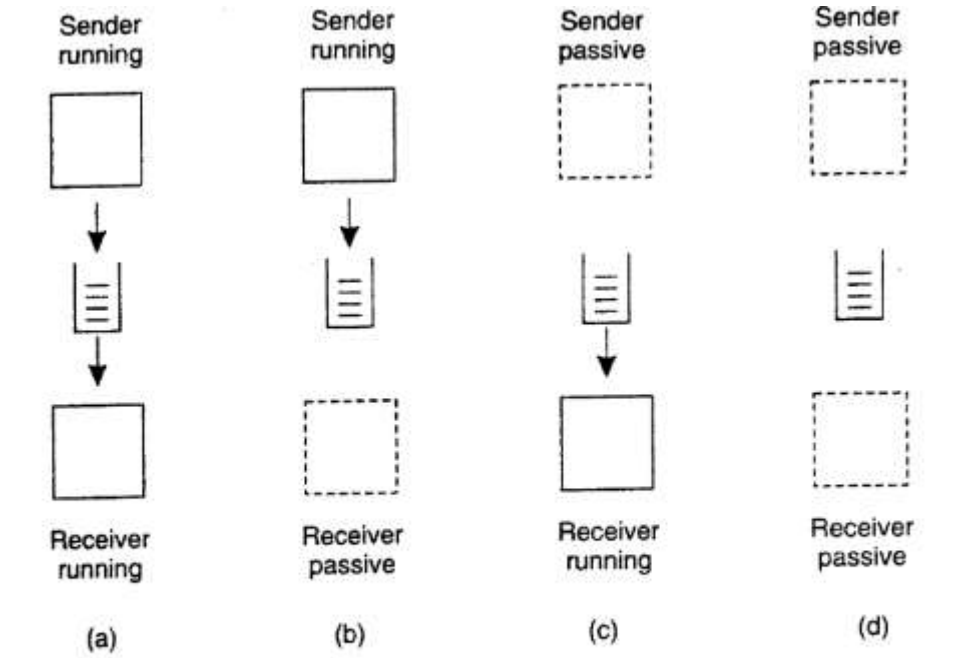


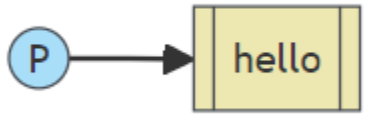
Figure 4-17. Four combinations for loosely-coupled communications using queues.

MQS primitives

Primitive	Meaning
Put	Append a message to a specified queue
Get	Block until the specified queue is nonempty, and remove the first message
Poll	Check a specified queue for messages, and remove the first. Never block
Notify	Install a handler to be called when a message is put into the specified queue

Figure 4-18. Basic interface to a queue in a message-queuing system.

Rabbit MQ example - sender




```
#!/usr/bin/env python
import pika

connection = pika.BlockingConnection(pika.ConnectionParameters('localhost'))
channel = connection.channel()
```

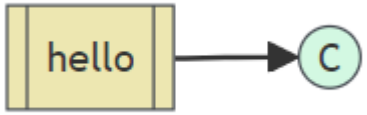
- Client RabbitMQ ⇒ modul Python pika
- Asigurăm existența cozii în care scriem mesajele (cu numele ‘hello’)

```
channel.queue_declare(queue='hello')
```

```
channel.basic_publish(exchange='',
                      routing_key='hello',
                      body='Hello World!')
print(" [x] Sent 'Hello World!'")
```

```
connection.close()
```

Rabbit MQ example - receiver




```
#!/usr/bin/env python
import pika

connection = pika.BlockingConnection(pika.ConnectionParameters('localhost'))
channel = connection.channel()
```

- Se realizează conexiunea identic
- Dacă nu cunoaștem a priori existența cozii create de sursă, asigurăm existența cozii în același fel

```
channel.queue_declare(queue='hello')
```

```
def callback(ch, method, properties, body):
    print(f" [x] Received {body}")
```

```
channel.basic_consume(queue='hello',
                      auto_ack=True,
                      on_message_callback=callback)
```

```
print(' [*] Waiting for messages. To exit press CTRL+C')
channel.start_consuming()
```

```
if __name__ == '__main__':
    try:
        main()
    except KeyboardInterrupt:
        print('Interrupted')
        try:
            sys.exit(0)
        except SystemExit:
            os._exit(0)
```

Rabbit MQ example

Execuție pe receiver:

```
python receive.py
# => [*] Waiting for messages. To exit press CTRL+C
```

```
# => [*] Waiting for messages. To exit press CTRL+C
# => [x] Received 'Hello World!'
```

Execuție pe sender:

```
python send.py
# => [x] Sent 'Hello World!'
```

https://www.rabbitmq.com/tutorials

Cuprins

- Ceasuri
- Algoritmi sincroni
- Alegere lider pe inel (sincron)

Ceasuri

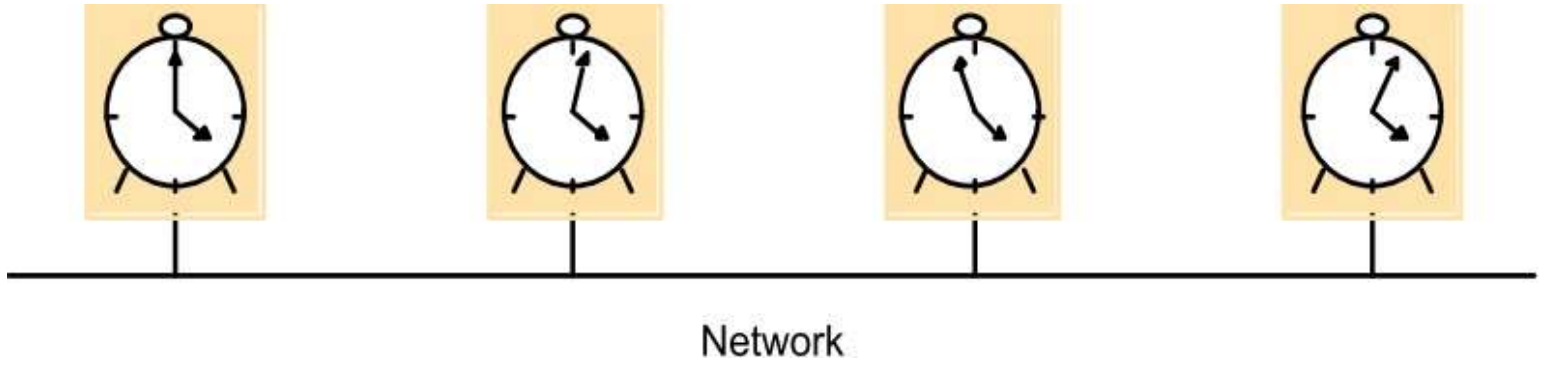
Timp

- Ceasurile fizice în computere sunt realizate prin contorizare
  - Ceasuri atomice: drift 1s/150 milioane de ani
  - Ceasuri de sistem
  - Ceasuri de timp real: alimentate prin baterie (funcționează chiar dacă sistemul este oprit)

- $h(t)$ *rata (viteza) ceasului hardware*
- $H(t) = \int_0^t h(\tau)\tau$ valoarea ceasului hardware
- Ceas logic (registru): $C(t) = \alpha H(t) + \beta$
- $C(t)$ este un scalar crescător și se actualizează prin citiri ale lui $H(t)$

Timp

- In SD, ceasurile locale sunt **decalate** și **întârziate**
- **Decalaj** între nodurile (i, j) la momentul t: $|C_i(t) - C_j(t)|$
- **Întârziere** între nodurile (i, j) la momentul t: $|\frac{d}{dt}C_i(t) - \frac{d}{dt}C_j(t)|$

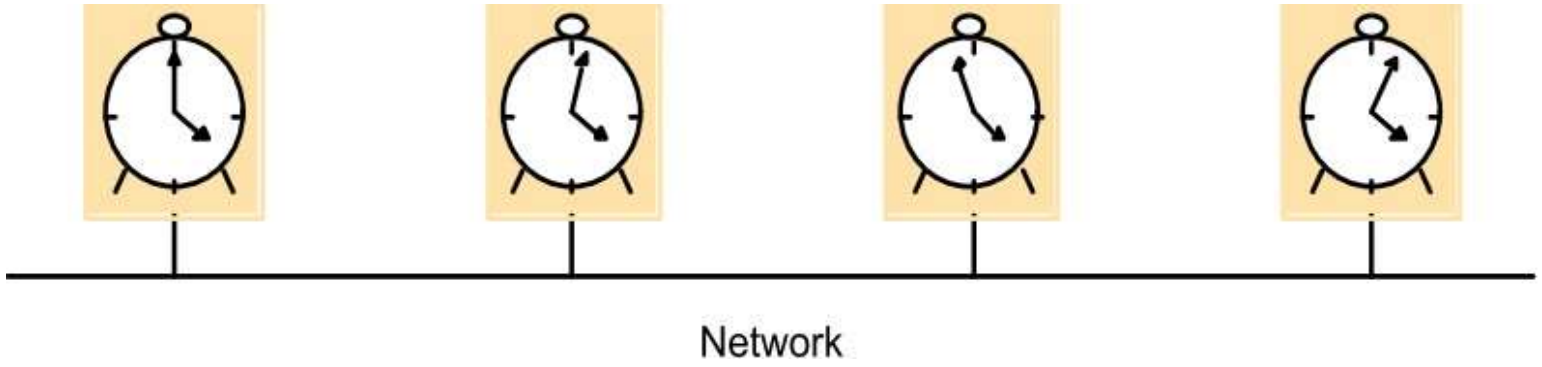


Problema sincronizării

Folosind referințe externe sau interne, problema sincronizării ceasurilor se reduce la asigurarea relației (de precizie):

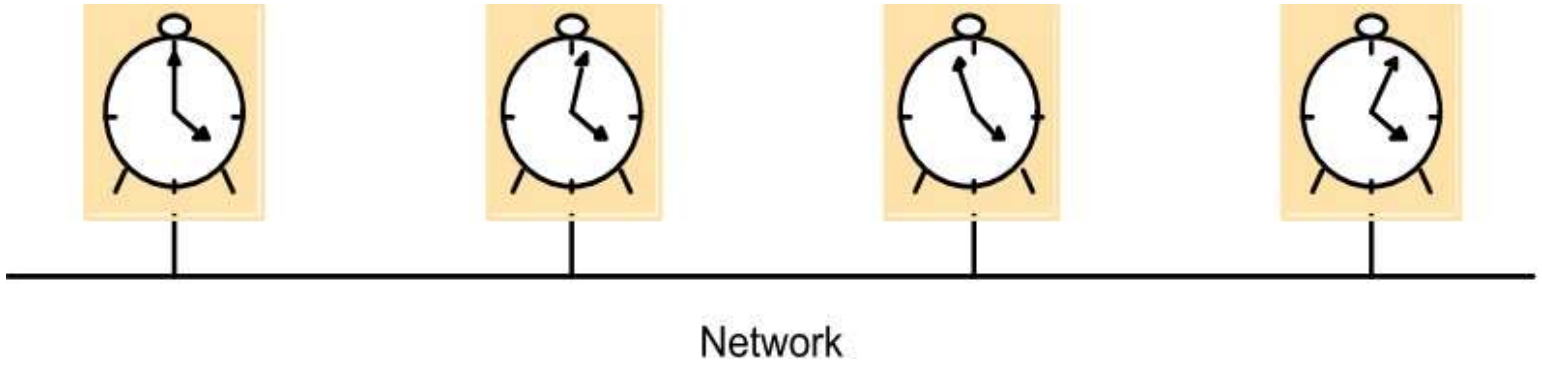
$$\left|C_i(t) - C_j(t)\right| \leq \rho \qquad \forall t \geq t_0, \forall i, j.$$

- Proprietate globală atinsă prin operații la nivel local!



Problema sincronizării

- Sincronizare externă: referința este un ceas absolut extern e.g. Coordinate Universal Time (UTC), GPS etc.
  - Algoritmul lui Cristian
- Sincronizare internă: referința este o valoare stabilită în rețea
  - Algoritmul de Medie (Berkeley Algorithm)



UTC

UTC este standardul primar de timp în lume, transmis prin: radio, linii telefonice line, satelit (GPS) etc.

Protocoalele populare de sincronizare folosesc referința UTC și urmăresc să asigure relația (acuratețe):

$$|S(t) - C_i(t)| < \rho \quad \forall i, \forall t$$

Dacă asigurăm acuratețea ρ atunci ce precizie garantăm?

Algoritmul lui Cristian

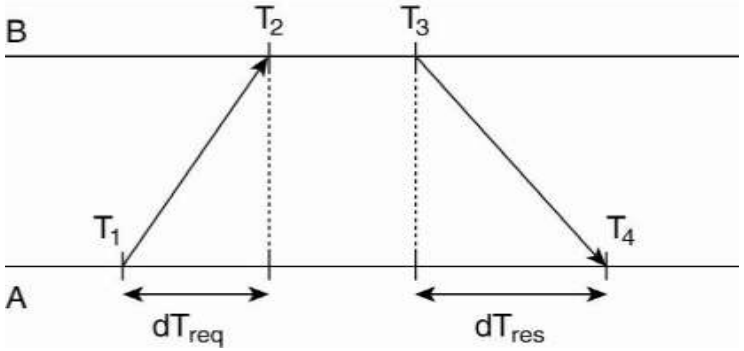
- Ipoteza 1: Întârzieri pe comunicație simetrice și mărginite (rețele LAN)
- Ipoteza 2: Există un nod de referință (pasiv) R cu unicul rol de a furniza referința
- Nodurile care nu sunt referința se vor sincroniza cu ceasul referinței

Algoritmul lui Cristian

Procedura AC: Nodul P cere periodic valoarea timpului de la R:

- T_1: send request; T_4: primește reply
- P primește val. T_2 și T_3 de la R, ajustează $C(t) = T_3 + T_{res}$ (T_{res} timp livrare mesaj)
- Folosește estimarea $T_{res} \approx \frac{T_{req}+T_{res}}{2}$

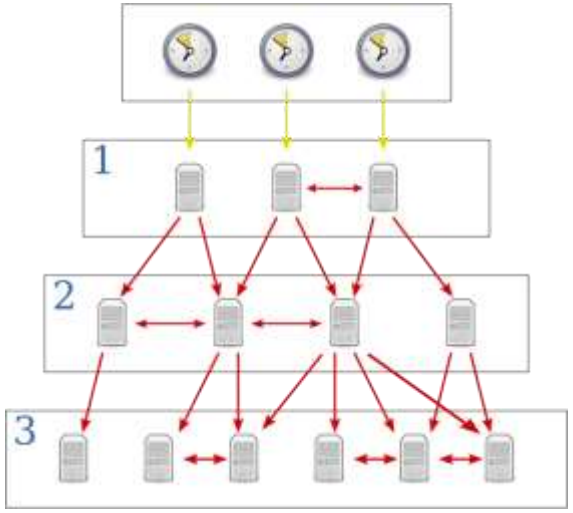
$$T_{res} = \frac{(T_2 - T_1) + (T_4 - T_3)}{2}$$



Network Time Protocol (NTP)

- Implementare standard în rețele a alg. lui Cristian
- Peste multiple măsurători, calculăm T_{res} minim

- Ierarhie a serverelor de timp:
  - un TS de nivel k se sincronizează după unul de nivel $\leq k - 1$
  - Nivelul 0 este referința UTC



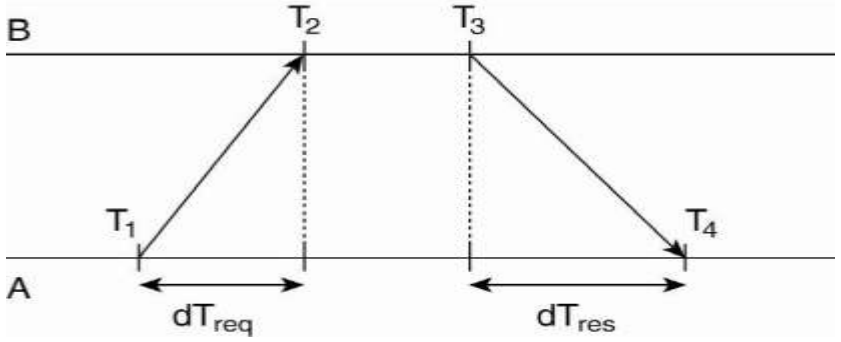
https://en.wikipedia.org/wiki/Network_Time_Protocol

Exemplu

La $5:08:15.100$, P trimite mesaj de request către R. La $5:08:15.900$, P primește răspuns de la B cu valoarea $5:08:25.300$ ($T_2 = T_3$)

- Care este valoarea ajustată a ceasului local $T_4 = C_P(t)$ după sincronizare?

$$T_{res} = \frac{(T_2 - T_1) + (T_4 - T_3)}{2}$$

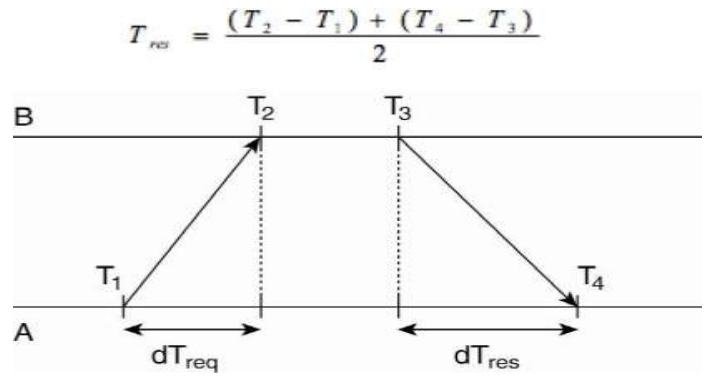


Exemplu

La $5{:}\,08{:}\,15.100$, P trimite mesaj de request către R. La $5{:}\,08{:}\,15.900$, P primește răspuns de la B cu valoarea $5{:}\,08{:}\,25.300$ ($T_2 = T_3$)

- Send req la $T_1 = 5{:}\,08{:}\,15.100$
- Primește răsp. la $T_4 = \ 5{:}\,08{:}\,15.900$
- Mesajul este$\mathrm{T}_3 = \mathrm{T}_2 = C_R(t) = \ 5{:}\,08{:}\,25.300$

- Durata totală $T_4 \ - \ T_1 = 800\ ms$
- Estimare: mesaj generat în urmă cu $400\ ms$
- Set $\mathrm{C_P(t)} = T_{serv} + 400 = 5{:}\,08.25.700$

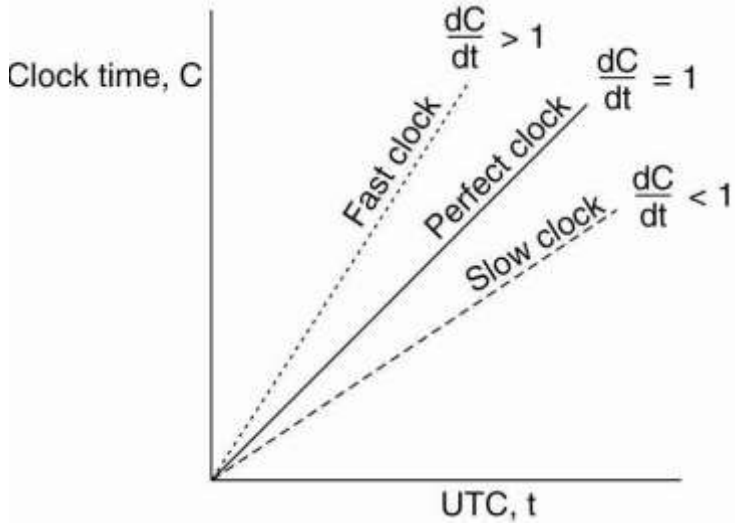


Sincronizare internă

Sincronizarea internă a ceasurilor locale în DS presupune (precizie)

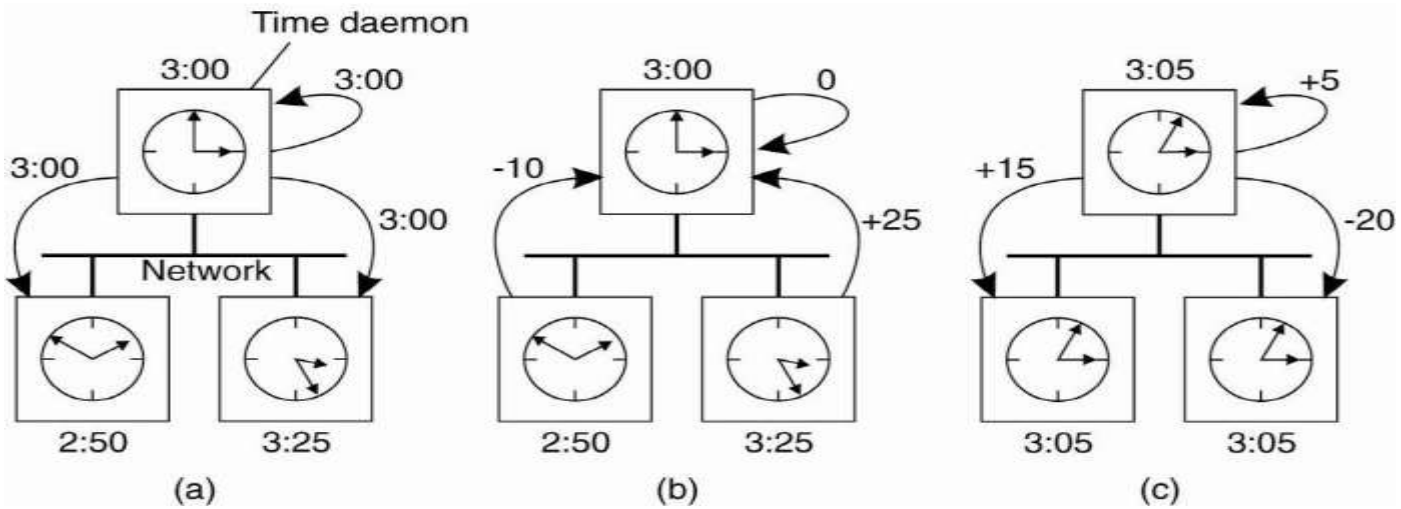
$$\left|C_j(t) - C_i(t)\right| < \rho \quad \forall i, j, t$$

- Necesită algoritmi complet distribuiți
- Problema este una de consens distribuit
- Vom reveni la ea în cursurile următoare



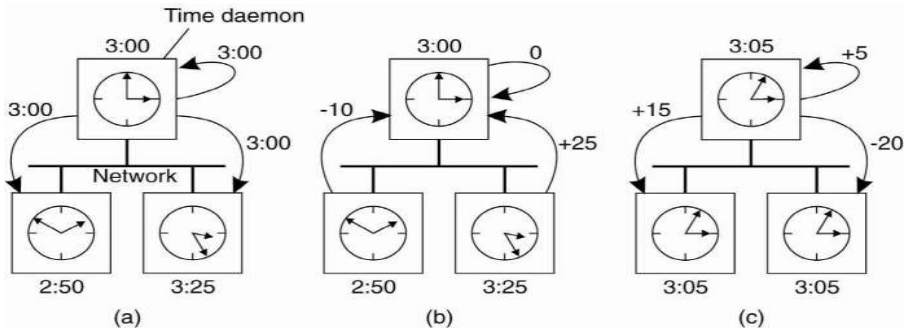
Algoritmul de medie (Berkeley Algorithm)

- Referința este unul din nodurile rețelei, ales eventual prin proceduri de leader-election
- Restul nodurilor urmăresc alinierea ceasurilor cu referința (consens)



Algoritmul de medie (Berkeley Algorithm)

- Referința este unul din nodurile rețelei, ales eventual prin proceduri de leader-election
- Restul nodurilor urmăresc alinierea ceasurilor cu referința (consens)
- Pe scurt: la iterația t
  - R difuzează valoarea $\mathrm{C_R}(t)$
  - P_i calculează întârzierea locală $\delta_\mathrm{i} = |\mathrm{C_R}\,(t) - C_i(t)|$ și răspunde lui R
  - R distribuie ajustările pentru $C_i\,(t)$



Alternative

1. If two machines don’t interact, there is no need to synchronize them (Leslie Lamport)

2. Dinamica întârzierii per link $\frac{T_{reg}+T_{res}}{2}$ depinde de mai mulți factori

3. Adesea, contează ca procesele să convină asupra **ordinii** evenimentelor, și nu asupra **timpului** la care au avut loc (vezi Ceasuri Logice).

Cuprins

- Ceasuri
- Algoritmi sincroni
- Alegere lider pe inel (sincron)

Algoritmi sincroni în SD

Un algoritm distribuit sincron reprezintă un set de operații de calcul/comunicație *executat în iterații/runde* (contorizate de t) pe nodurile sistemului, cu scopul rezolvării unei sarcini concrete.

- Denumim starea locală a nodului i la momentul t cu $x_i(t)$. Scopul algoritmului este conducerea lui $x_i(t)$ către starea optimă $x_i(\infty)$

- La următorul moment de timp starea x_i suferă o transformare bazată pe pașii algoritmului și informația provenită de la vecini. Pe scurt,

$$x_i(t+1) := f_i(x(t))$$

unde $f_i(\cdot)$ reprezintă funcția de transformare asociată nodului i. Funcția $f_i(\cdot)$ definește însuși algoritmul SPMD.

Algoritmi sincroni în SD

La următorul moment de timp starea x_i suferă o transformare bazată pe pașii algoritmului și informația provenită de la vecini. Pe scurt,

$$x_i(t+1) := f_i(x(t))$$

unde $f_i(\cdot)$ reprezintă funcția de transformare asociată nodului i.

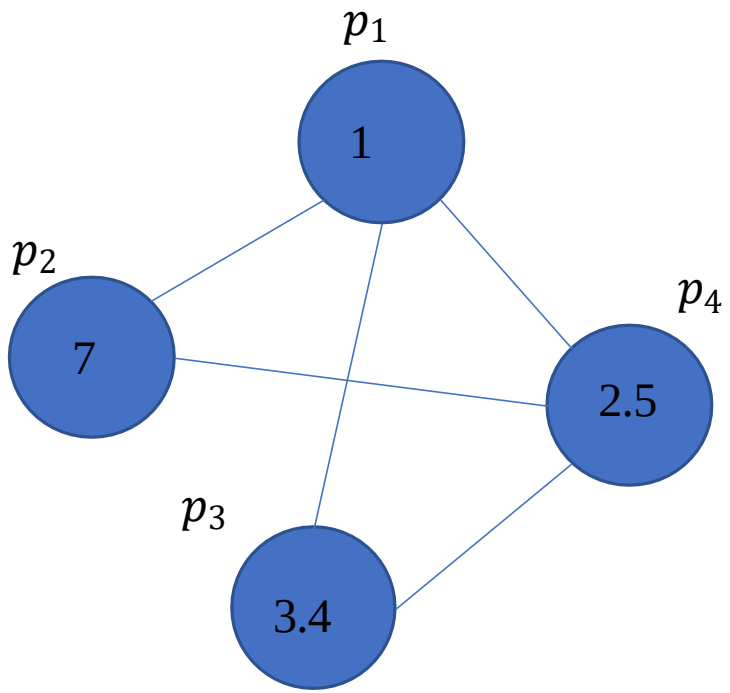
Funcția $f_i(\cdot)$ este definită în format SPMD și este compusă din una sau mai multe operații (e.g. aritmetice, numerice) asupra variabilelor locale din memoria nodului i.

Medie șir de numere reale

Problemă [Distributed averaging]: Vectorul $x(0)$ este distribuit peste n noduri, astfel încât $x_i(0)$ se află în memoria locală a nodului i .

Calculează distribuit media aritmetică $\mathrm{m} = \frac{\sigma_i \, x_i(0)}{n}$ a vectorului $x(0)$, încât la final $x_i(\infty) = m$.

$$x(0) = \begin{bmatrix} 1 \\ 7 \\ 3.4 \\ 2.5 \end{bmatrix}$$

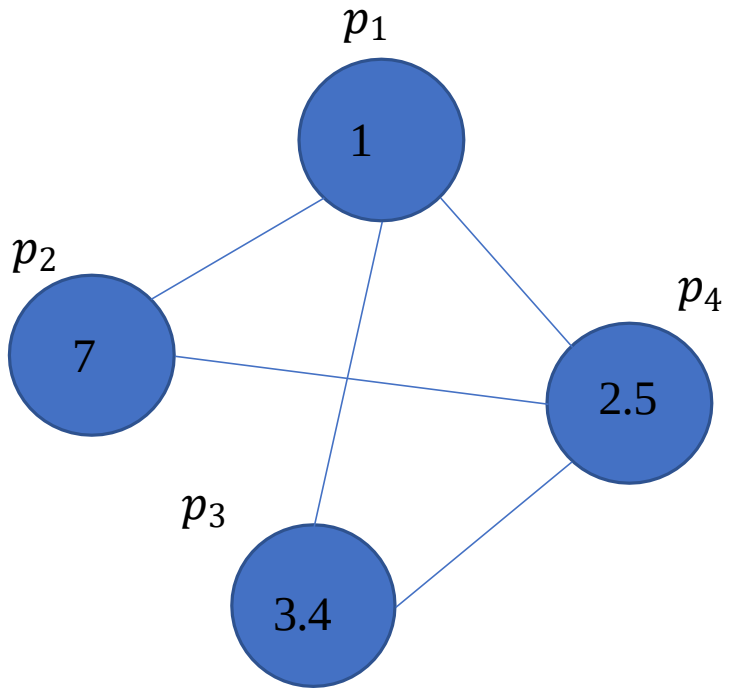


Medie șir de numere reale

Problemă [Distributed averaging]: Vectorul $x(0)$ este distribuit peste n noduri, astfel încât $x_i(0)$ se află în memoria locală a nodului i. Calculează distribuit media aritmetică $m = \sigma_i\, x_i(0)/n$ a vectorului $x(0)$, încât la final $x_i(\infty) = m$.

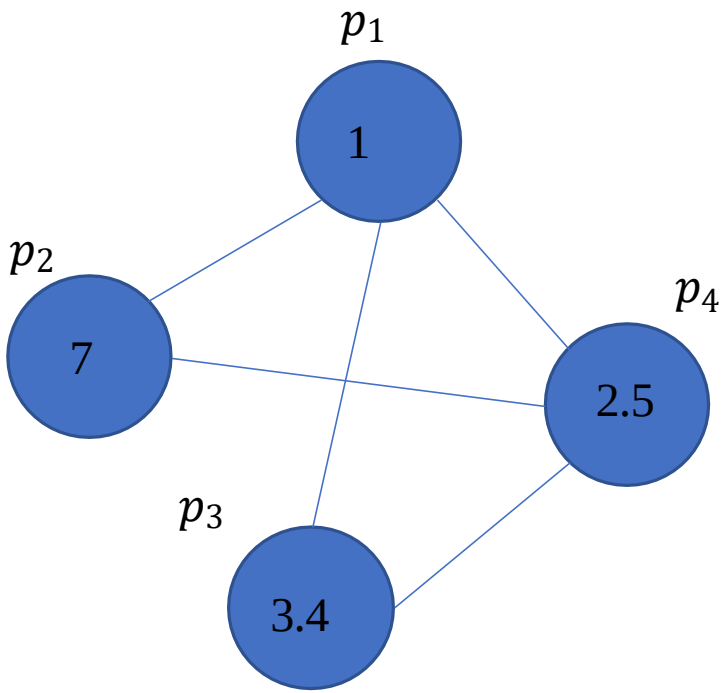
Algoritm: Alege $j \in N_i$ și actualizează $x_i(t+1) = f_i\big(x(t)\big) = \frac{x_i(t)+x_j(t)}{2}$

$$x(0) = \begin{bmatrix} 1 \\ 7 \\ 3.4 \\ 2.5 \end{bmatrix}$$



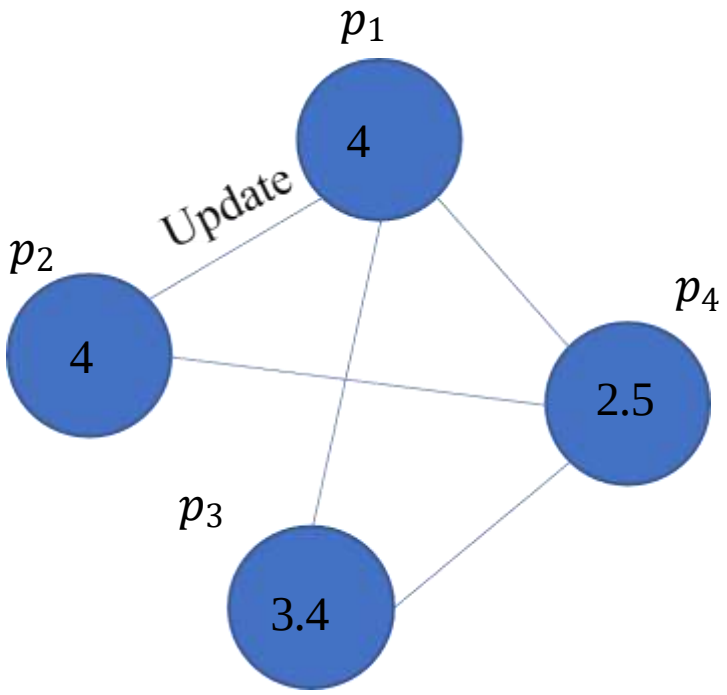
Medie șir de numere reale

$$x(0) = \begin{bmatrix} 1 \\ 7 \\ 3.4 \\ 2.5 \end{bmatrix}$$



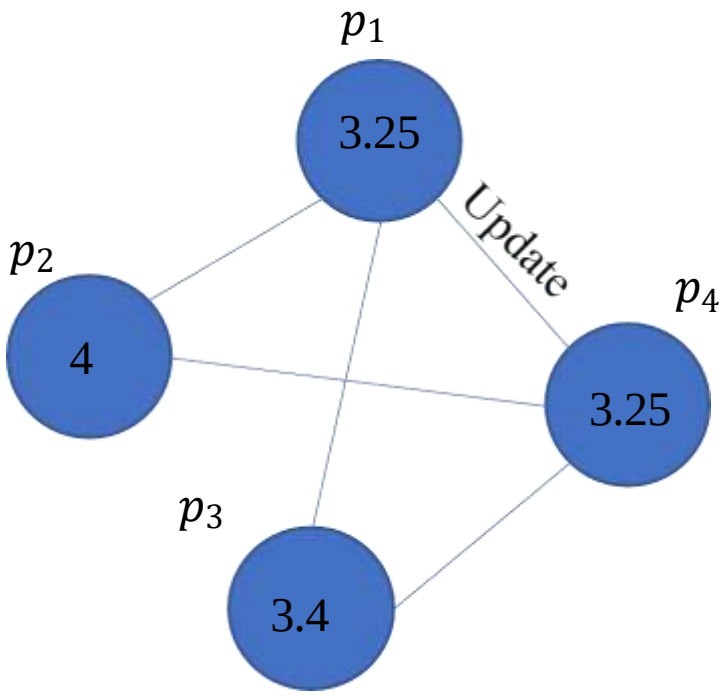
Medie șir de numere reale

$$x(1) = \begin{bmatrix} 4 \\ 4 \\ 3.4 \\ 2.5 \end{bmatrix}$$



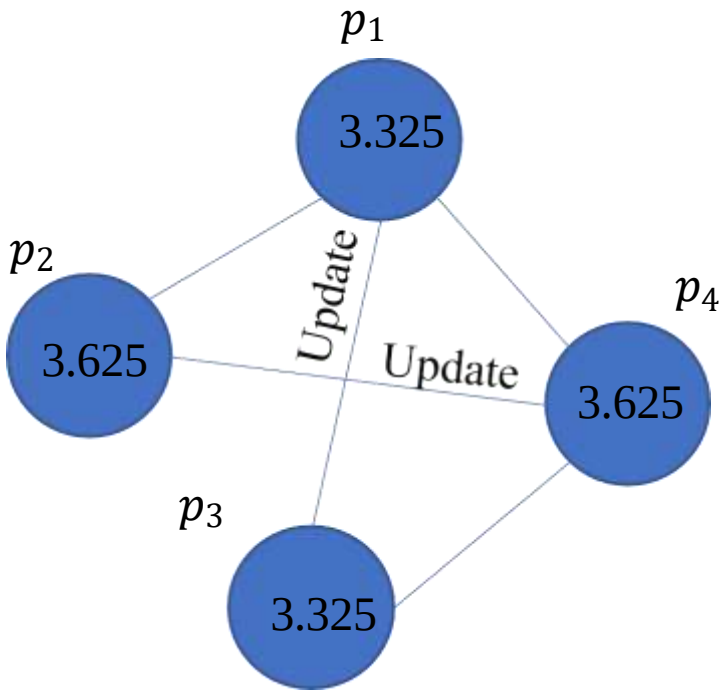
Medie șir de numere reale

$$x(2) = \begin{bmatrix} 3.25 \\ 4 \\ 3.4 \\ 3.25 \end{bmatrix}$$



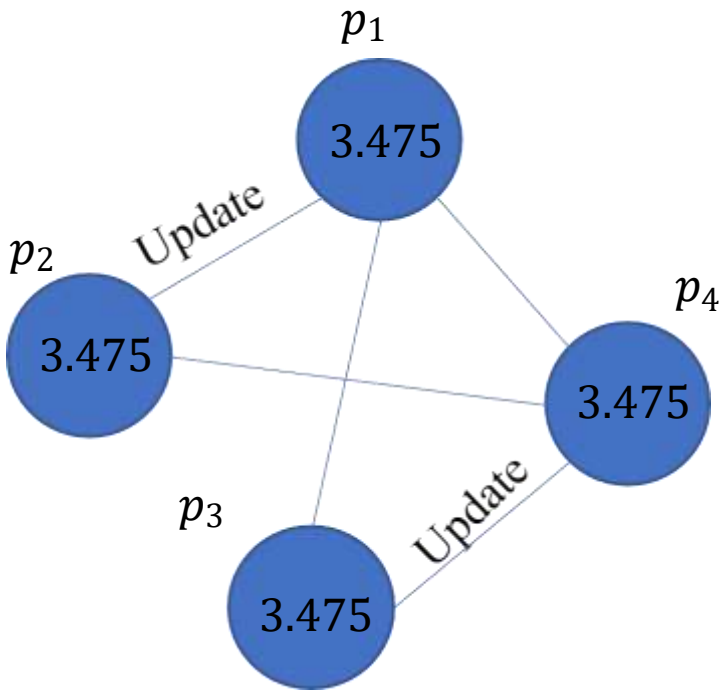
Medie șir de numere reale

$$x(4) = \begin{bmatrix} 3.325 \\ 3.625 \\ 3.325 \\ 3.625 \end{bmatrix}$$



Medie șir de numere reale

$$x(6) = \begin{bmatrix} 3.475 \\ 3.475 \\ 3.475 \\ 3.475 \end{bmatrix} = x^*$$



Algoritmi Sincroni în SD

La finalul iterației t fiecare nod finalizează calculul local, și comunicația cu vecinii, specifice iterației t

$$x(t+1) := \begin{bmatrix} x_1(t+1) \\ \dots \\ x_n(t+1) \end{bmatrix} = \begin{bmatrix} f_1(x(t)) \\ \dots \\ f_n(x(t)) \end{bmatrix} = F(x(t))$$

- Există o margine superioară pe timpul de comunicație între oricare două noduri.
- Adesea implementarea unui criteriu de oprire este o operație dificilă!

Alegere Lider (Leader Election)

În multe aplicații este necesară alegerea unui nod pentru operații particulare (e.g. difuzare, distribuție, master-slave).

Fiecare nod are un ID **unic**, ales dintr-un spațiu total ordonat.

Convenție: Lider = nodul cu ID-ul maxim.

Algoritmii de LE realizează *de facto* calculul distribuit al funcției

$$\max\{id_1, id_2, \dots, id_n\}$$

Alegere Lider (Leader Election)

Starea locală a nodului i specifică calitatea de lider/non-lider:

$$x_i(t) \in \{lider, non-lider\}$$

Funcția de transformare asociată nodului i:

$$f_i\big(\{x_j(t) | j \in N_i\}\big)$$

decide dacă la iterația curentă nodul i devine sau nu lider.

Funcția f_i se reduce la una sau mai multe operații asupra memoriei locale M_i a nodului P_i.

Alegere Lider (Leader Election)

Starea locală a nodului i specifică calitatea de lider/non-lider:

$$x_i(t) \in \{lider, non-lider\}$$

Funcția de transformare asociată nodului i:

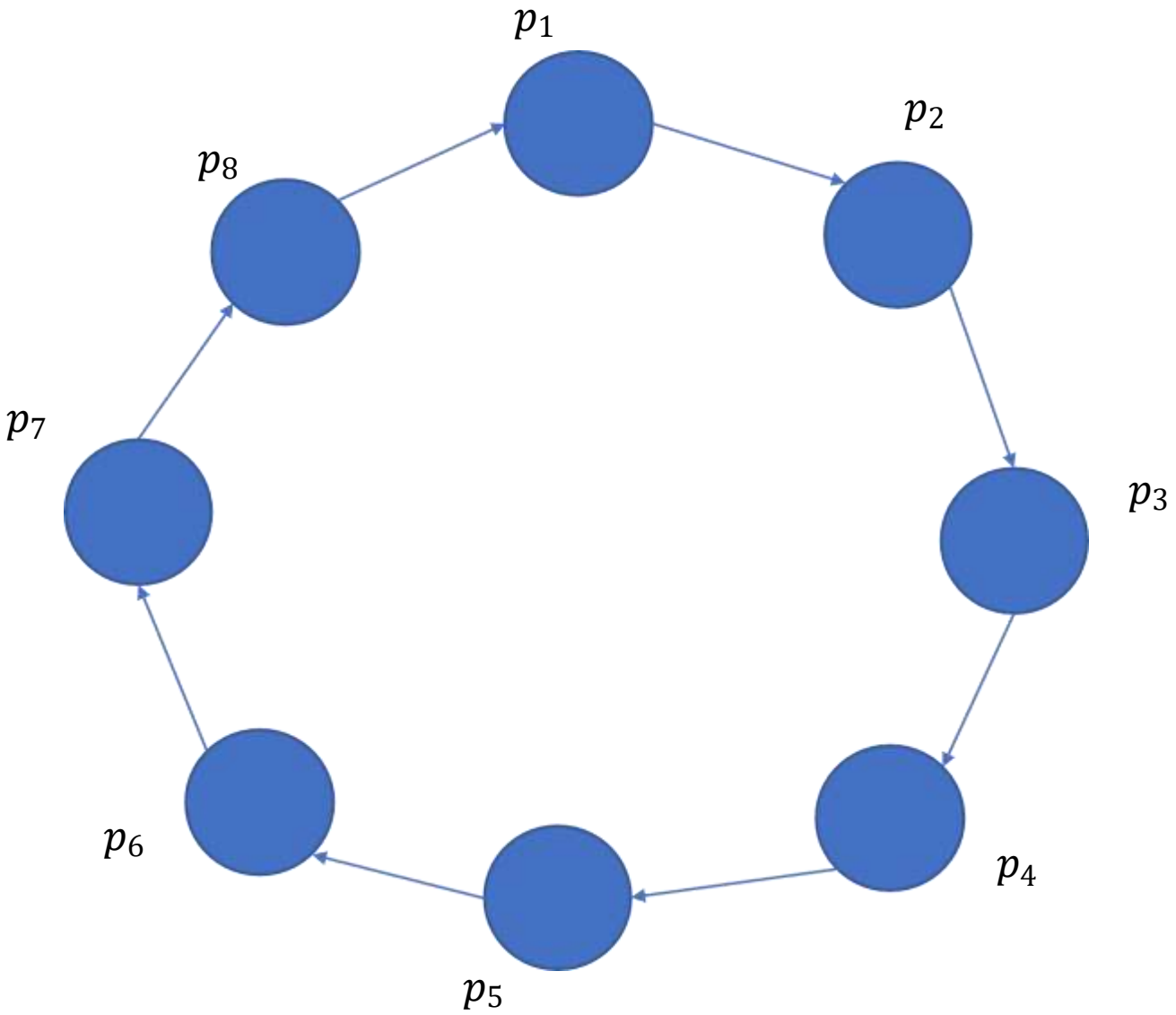
$$f_i\left(\{x_j(t) | j \in N_i\}\right)$$

decide dacă la iterația curentă nodul i devine sau nu lider.

Asimptotic, soluția problemei este aducerea sistemului în starea:

$$x_i^* = \begin{cases} lider, & i = argmax_j\ id_j \\ non-lider, & altfel \end{cases}$$

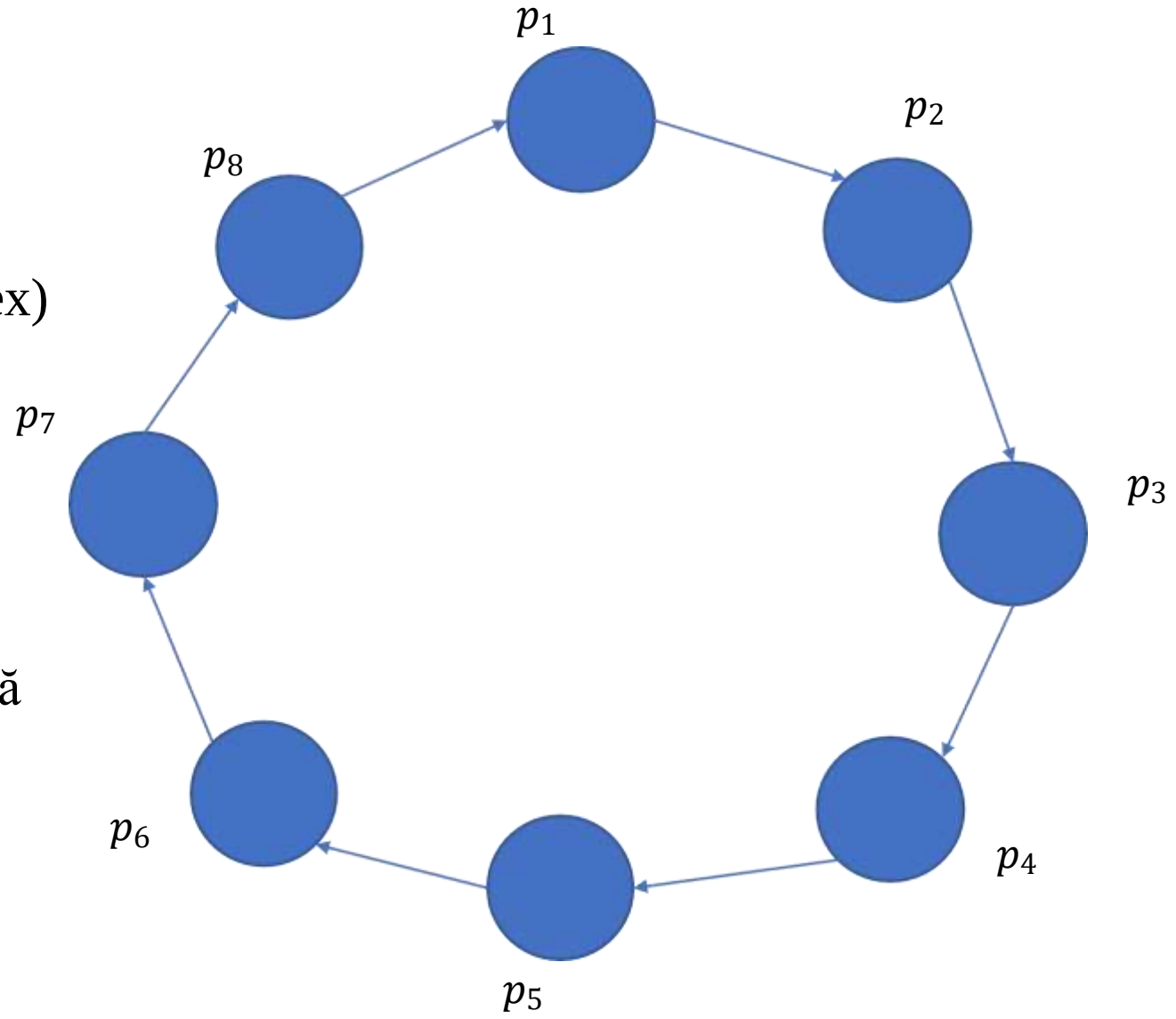
Alegere Lider (AL)



Alegere Lider (AL)

Ipoteze: P_i cunoaște
1. Topologie inel unidirecțional
2. Poziția proprie relativă în topologie (index)
3. Numărul de noduri n

Problema: Specifică un program SPMD (f_i) astfel încât, într-un număr minim de iterații să asigurăm convergența stării globale la starea optimă, *i.e.* $x_i(T) = x_i^*$.



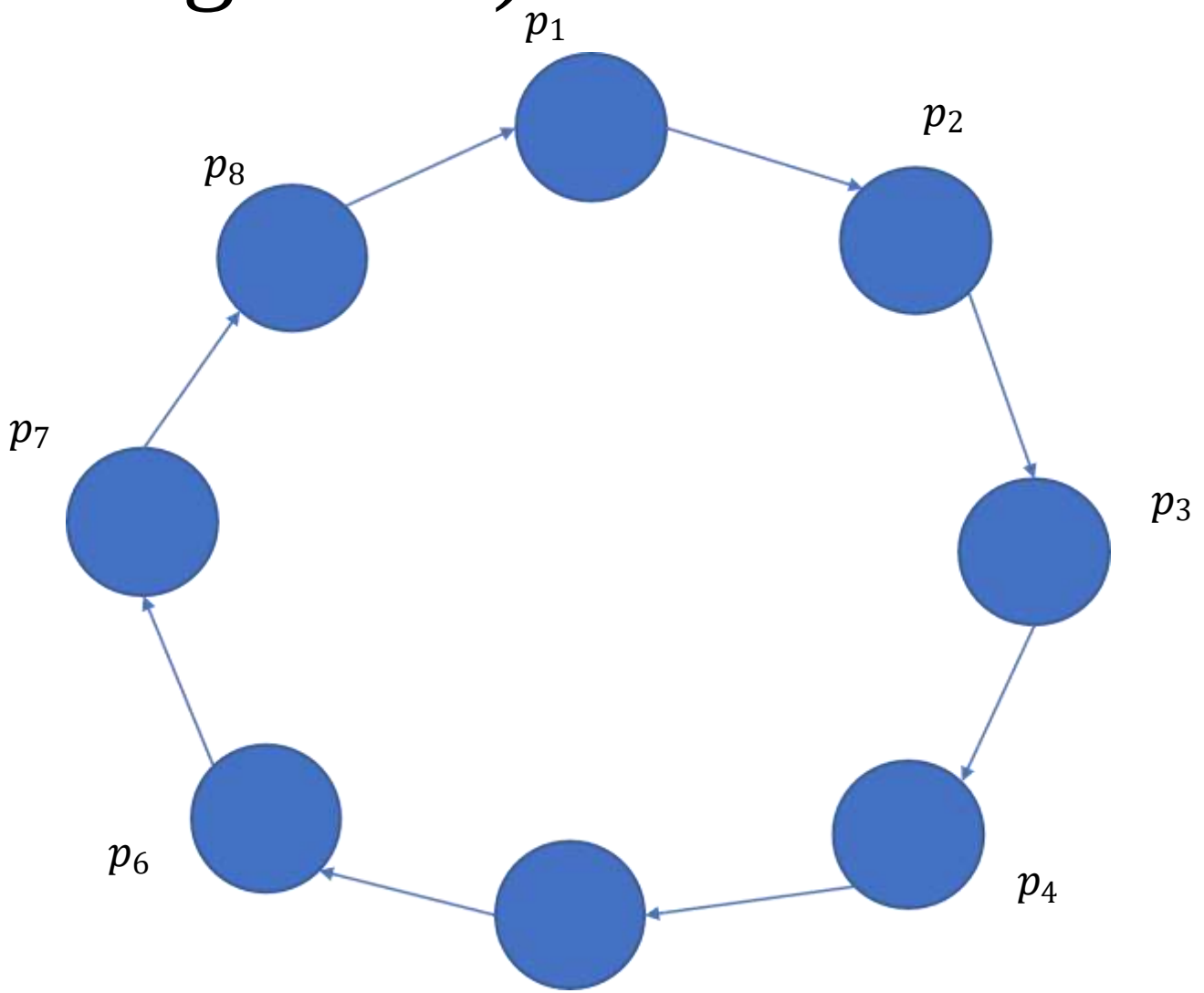
Alegere Lider (coordonate globale)

Algoritm **AlegeLiderInel**():

M_i: - int n (număr noduri)
- int i (index)
- int id (token)
- int id_aux, id_max (memorează id maxim)

% Faza I: max ID
- Calculează max$\{id_1, id_2, \ldots, id_n\}$
- Rezultatul va fi stocat într-un nod particular

% Faza II: Difuzare Max ID (Broadcast)
- Rezultatul este difuzat peste tot inelul
- Stările x_i sunt ajustate conform rezultatului



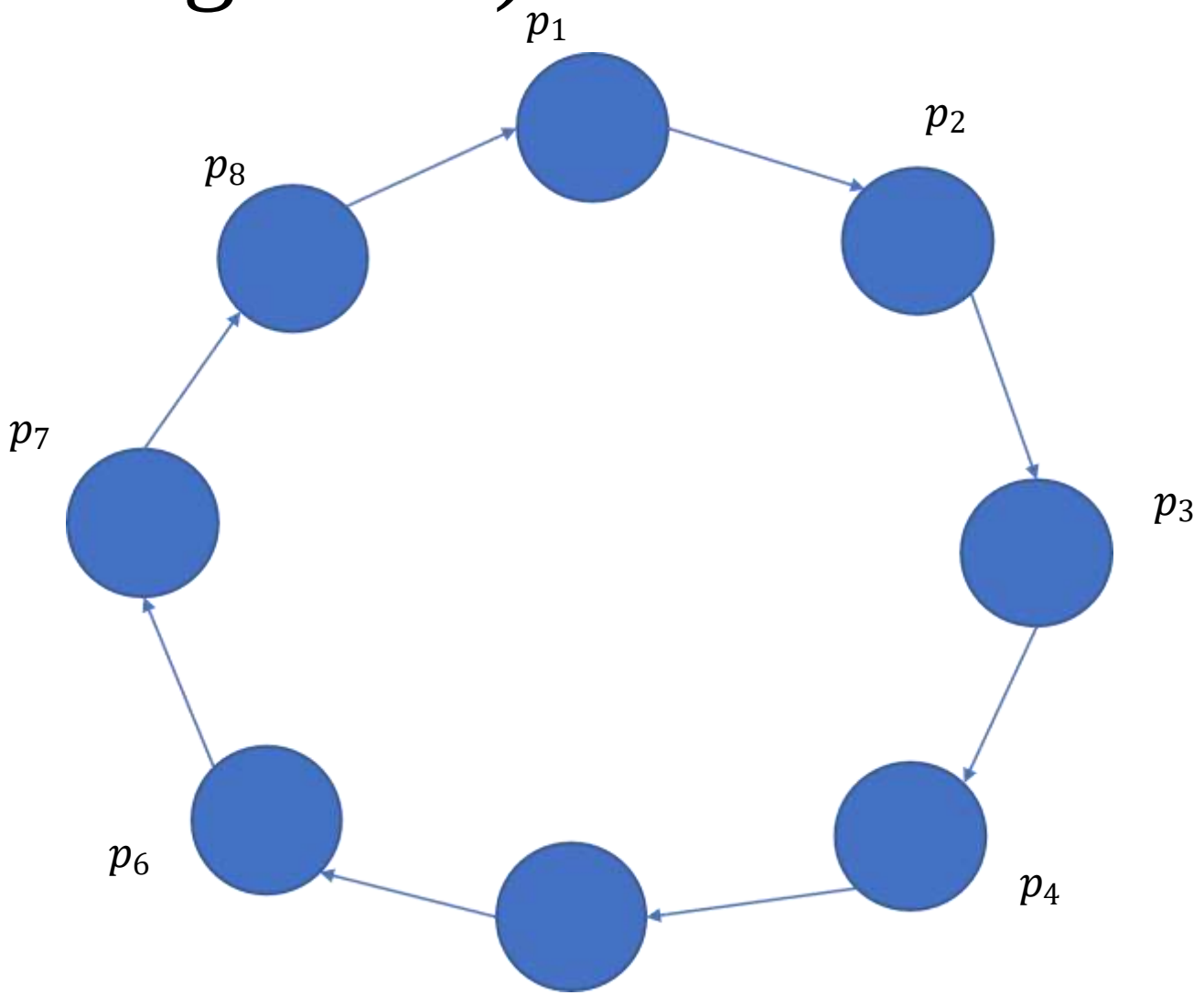
Alegere Lider (coordonate globale)

M_i: - int n (număr noduri)
- int i (index)
- int id (token)
- int id_aux, id_max (memorează id maxim)

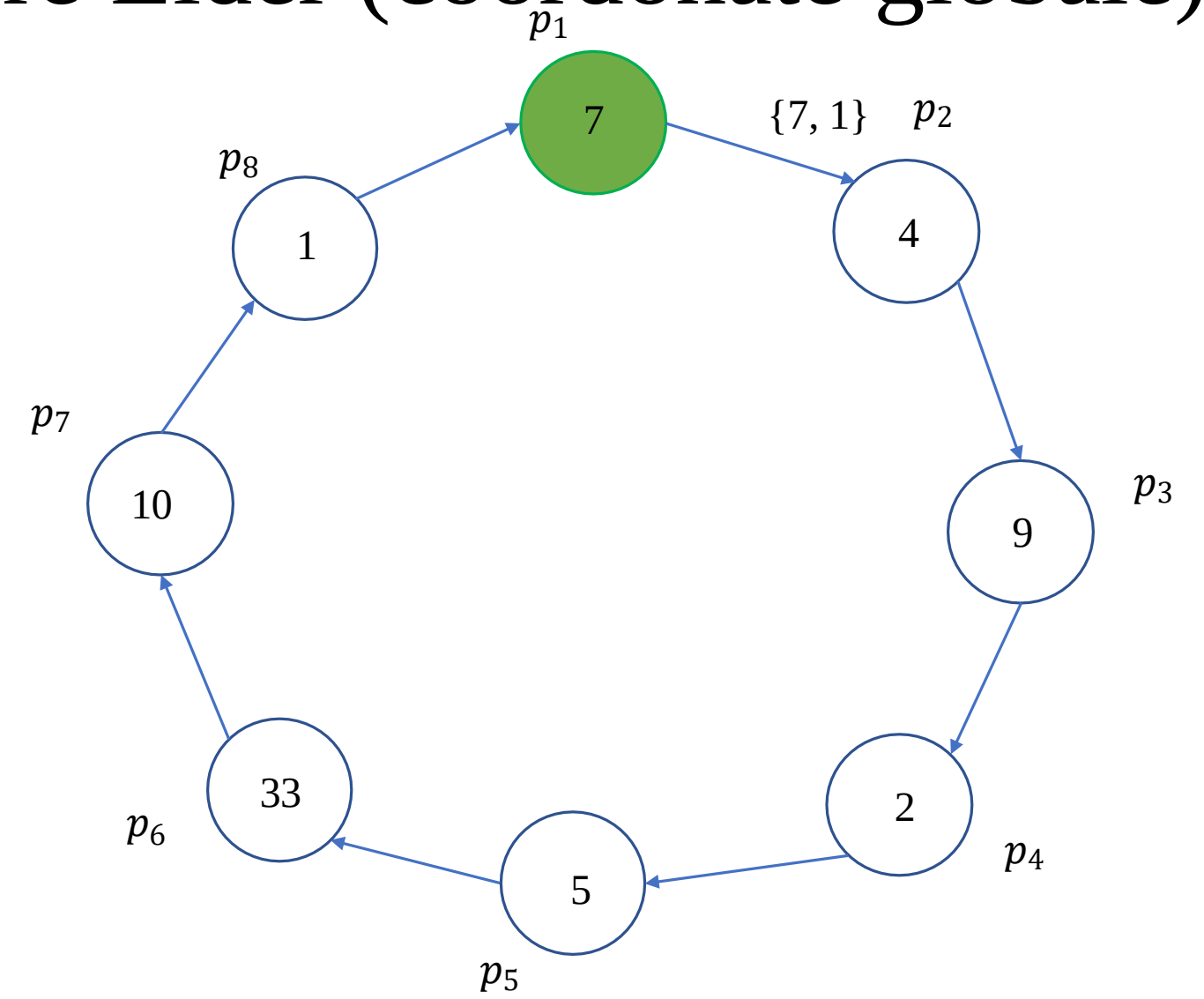
% Faza max ID

Functie transformare nod i $f_i()$:

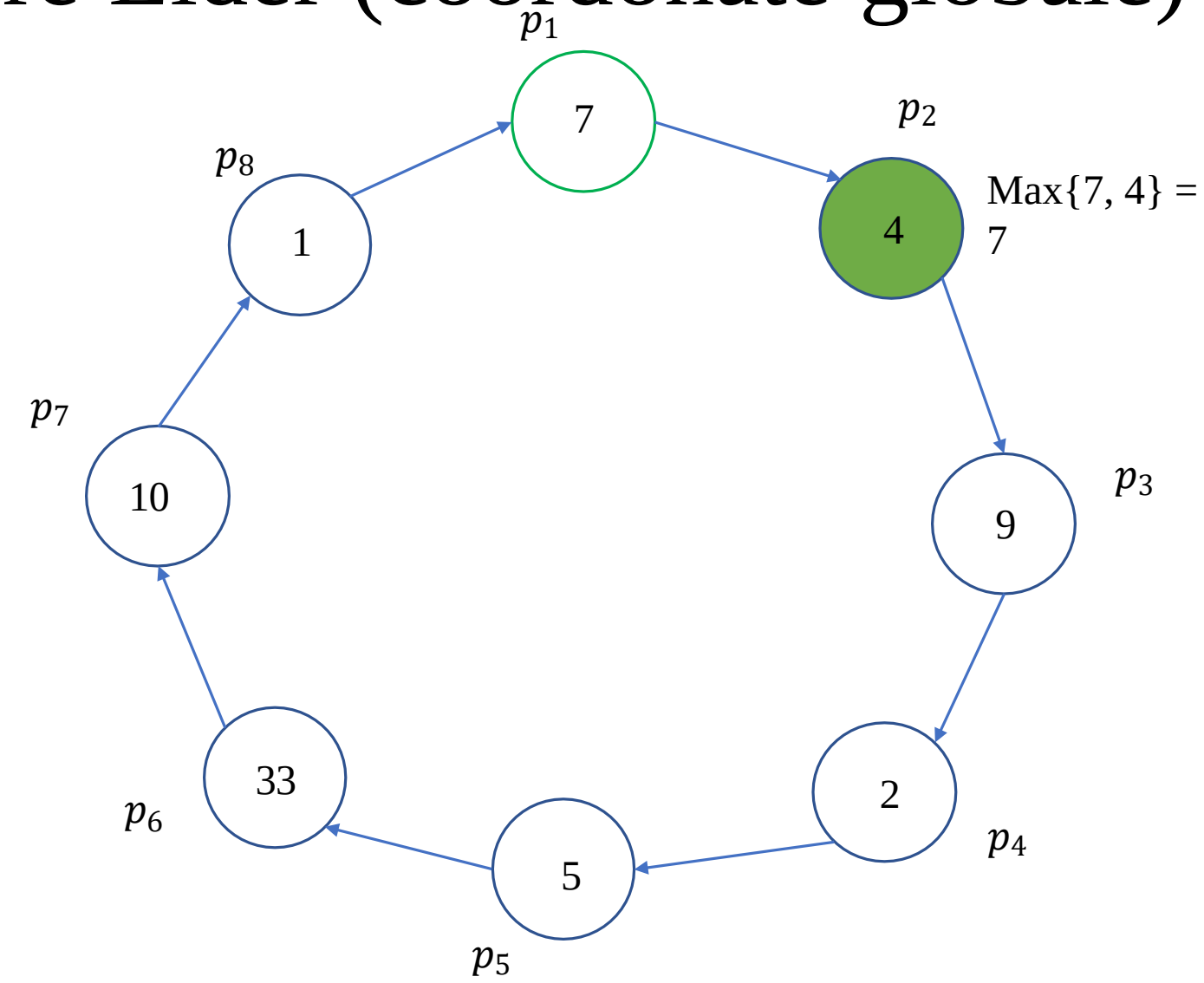
1. **If (i == 1):**
    1. send(id, 2);
    2. id_aux = recv(n);
2. **else:**
    1. id_aux = recv(i - 1 mod n);
    2. idmax = max(id, id_aux);
    3. send(idmax, i + 1 mod n);



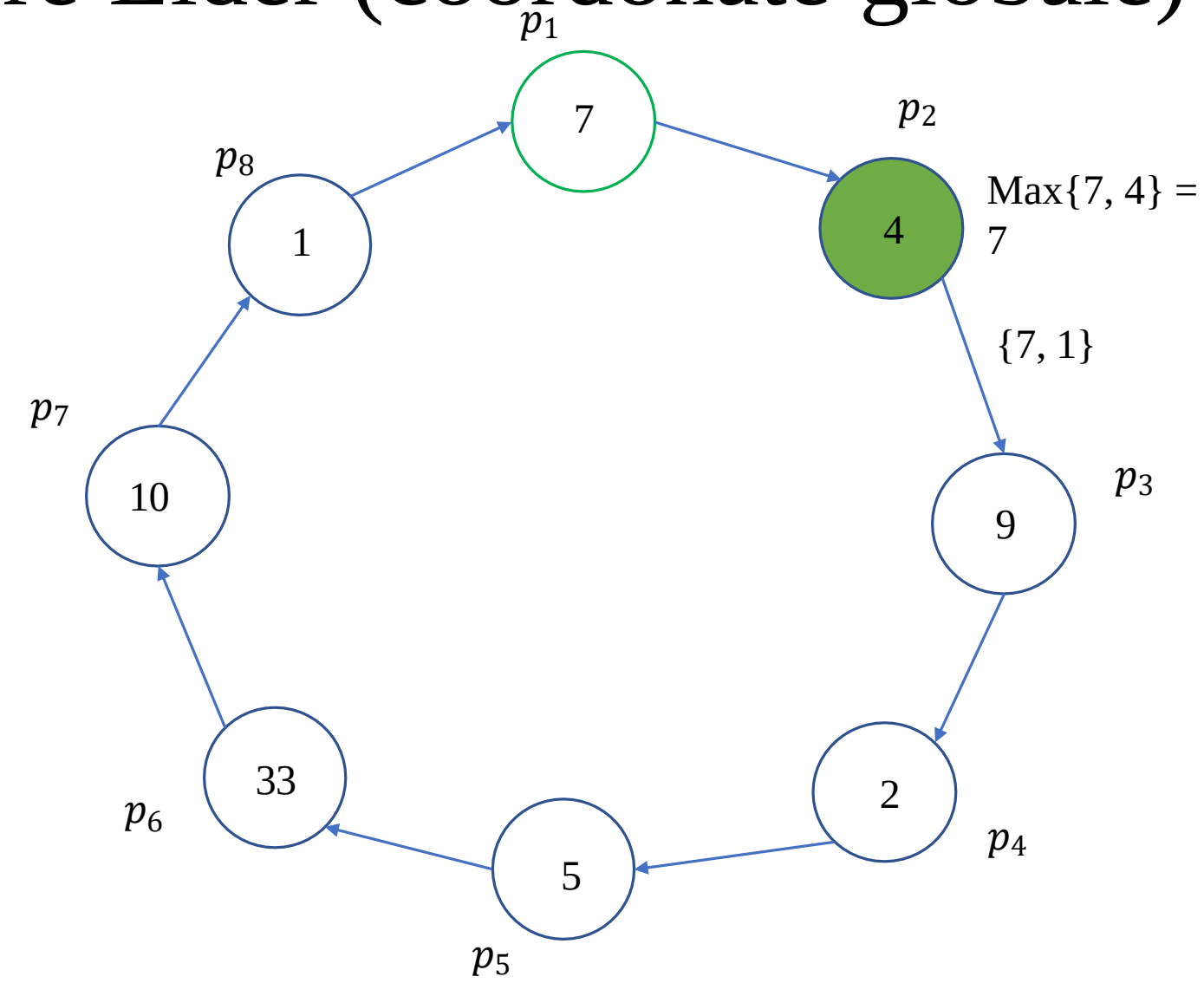
Alegere Lider (coordonate globale)



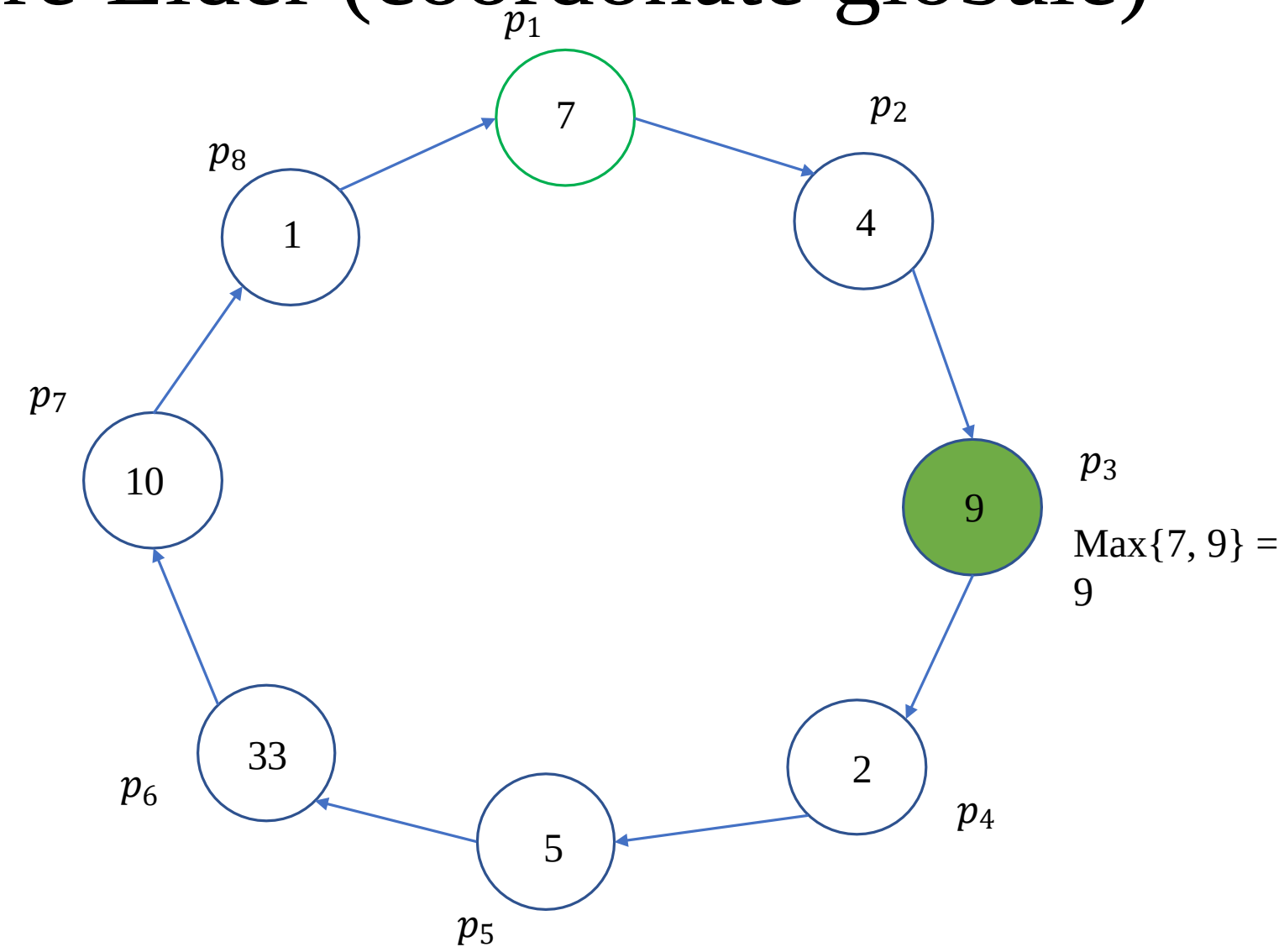
Alegere Lider (coordonate globale)



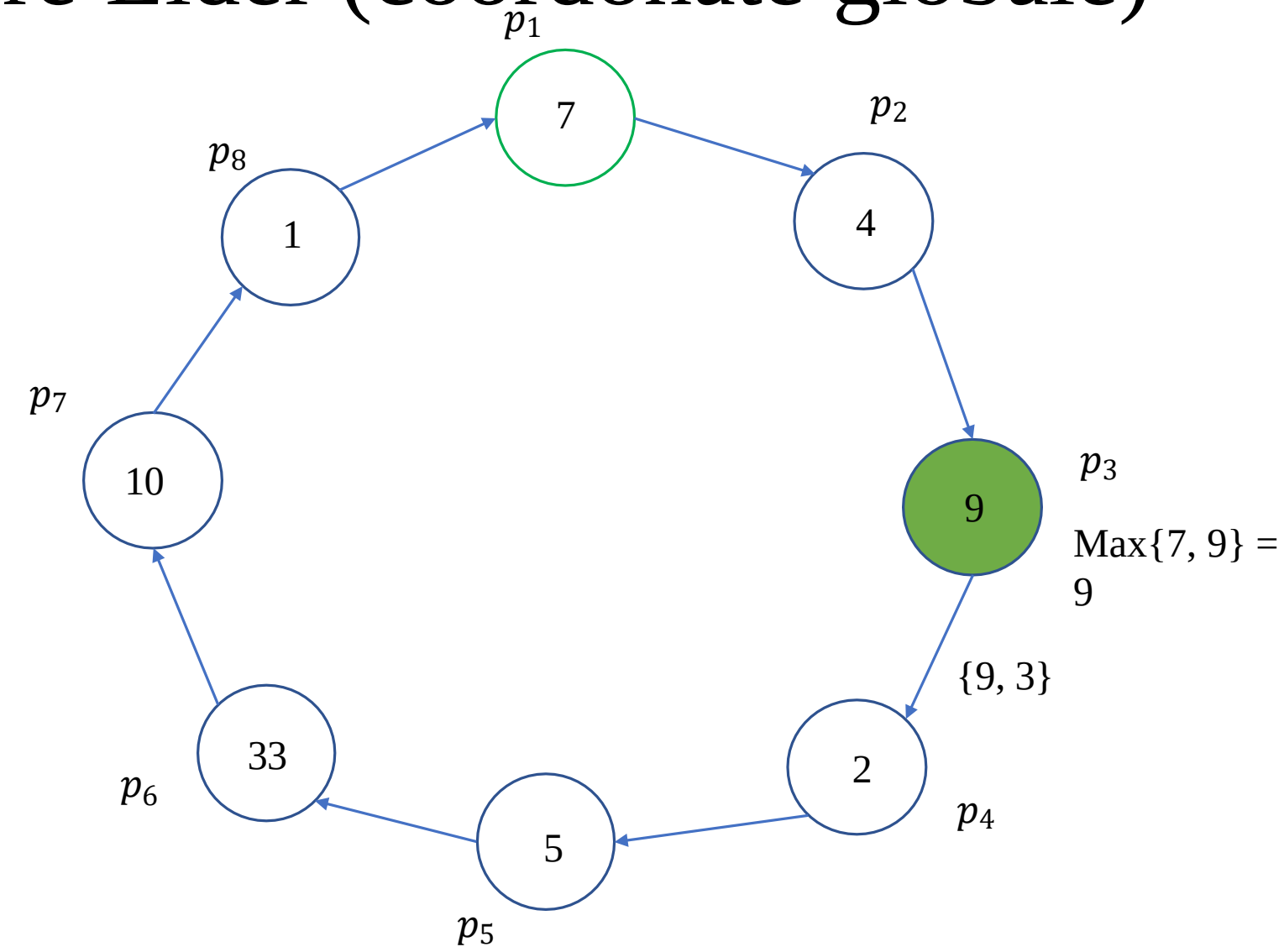
Alegere Lider (coordonate globale)



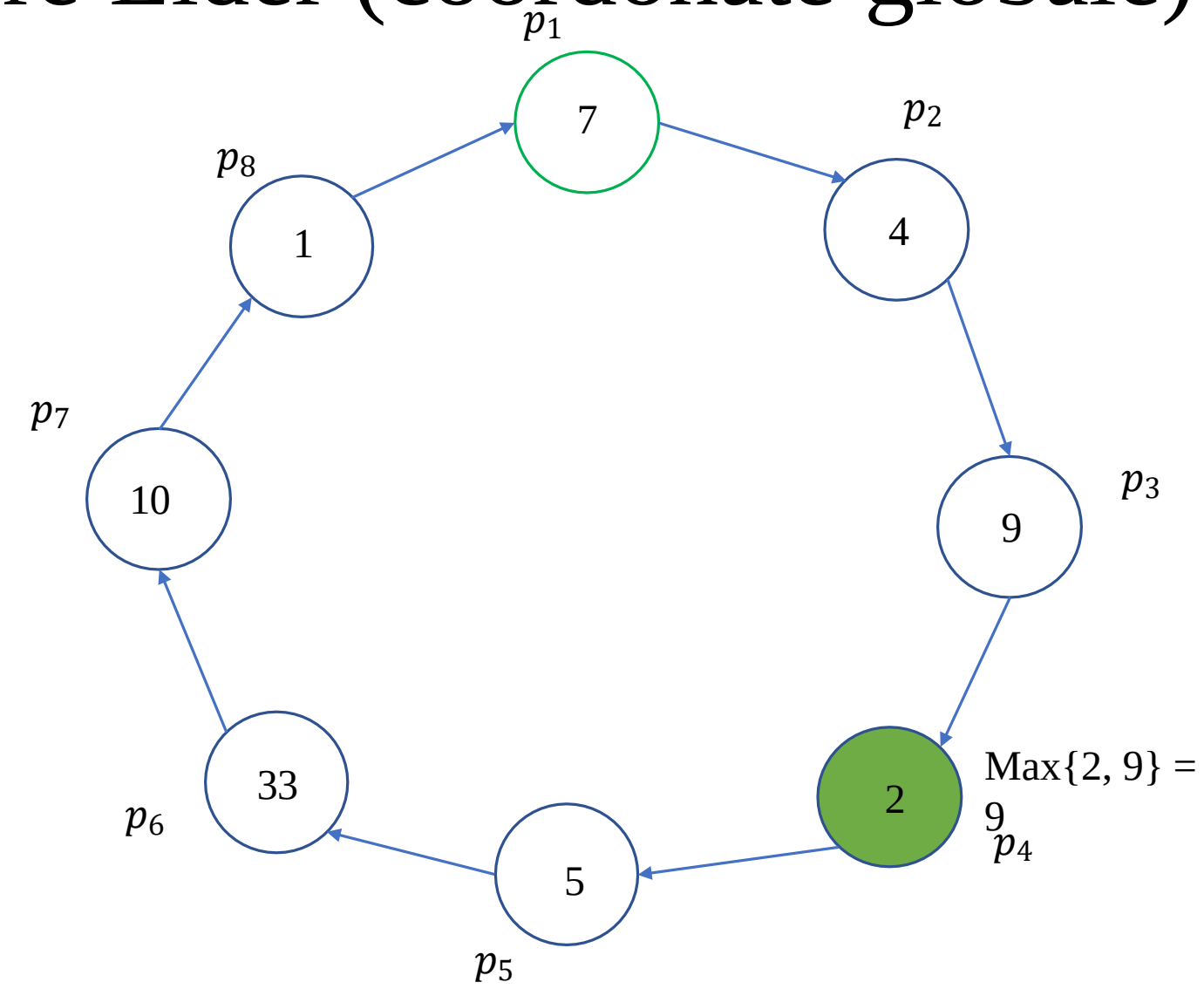
Alegere Lider (coordonate globale)



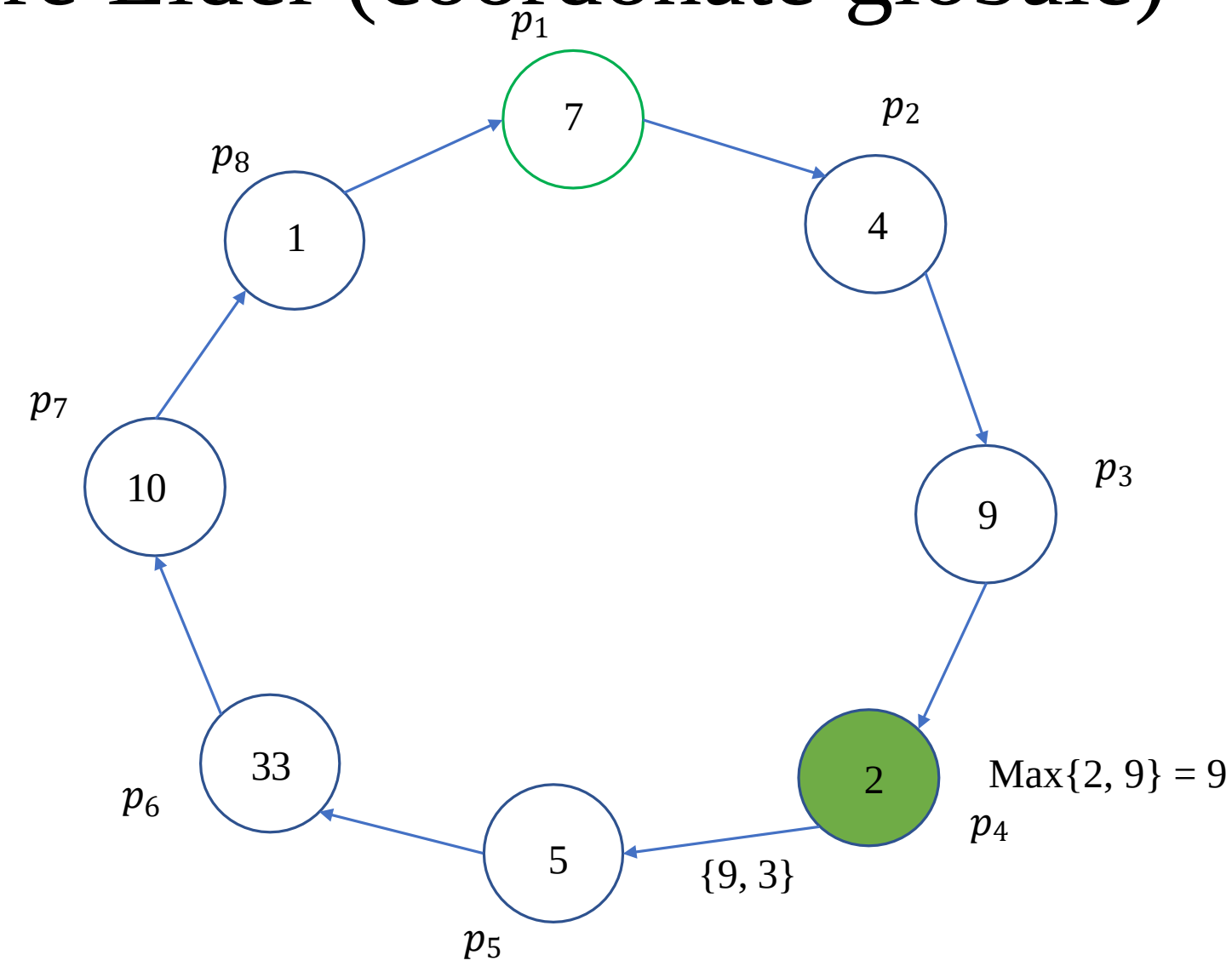
Alegere Lider (coordonate globale)



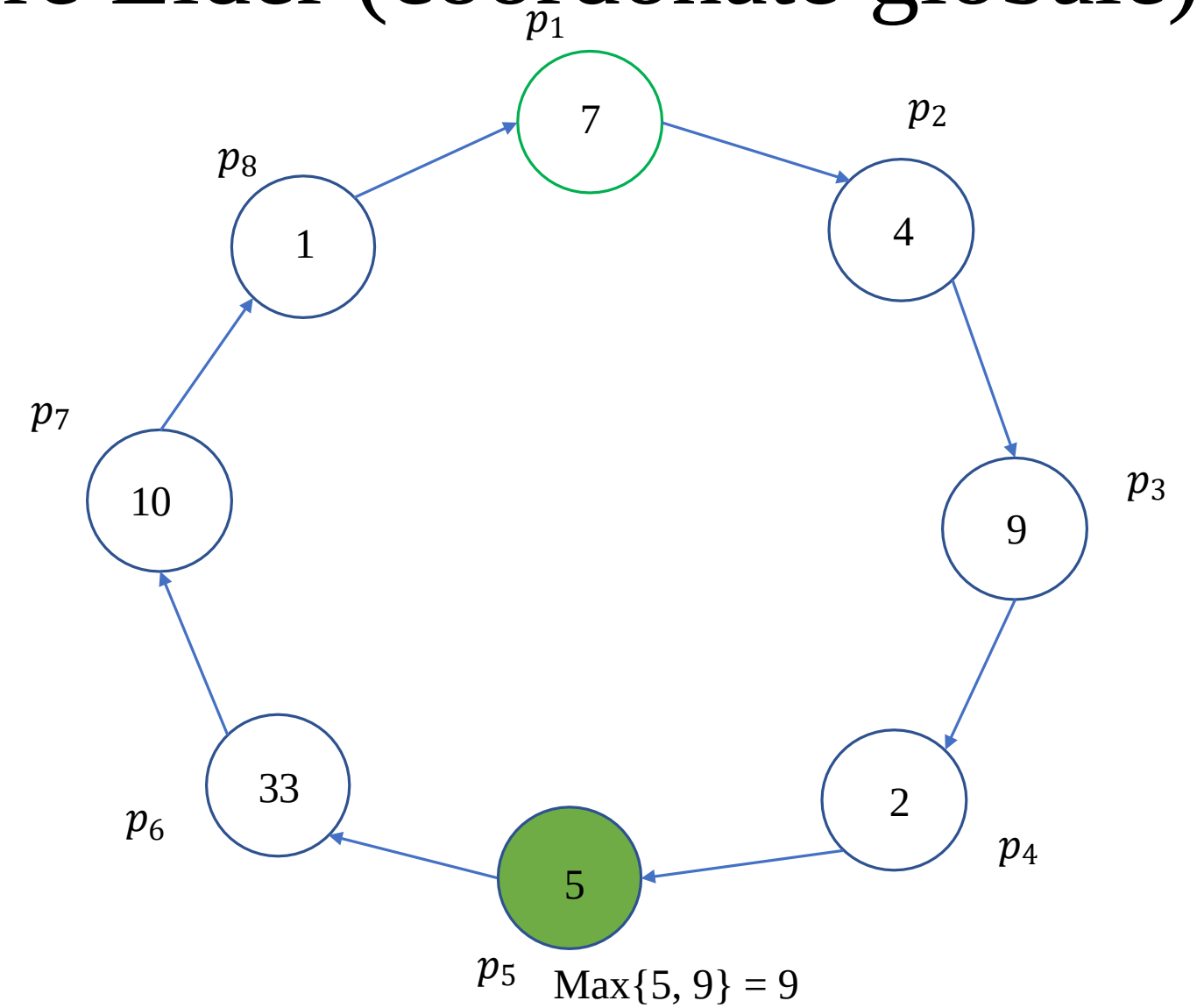
Alegere Lider (coordonate globale)



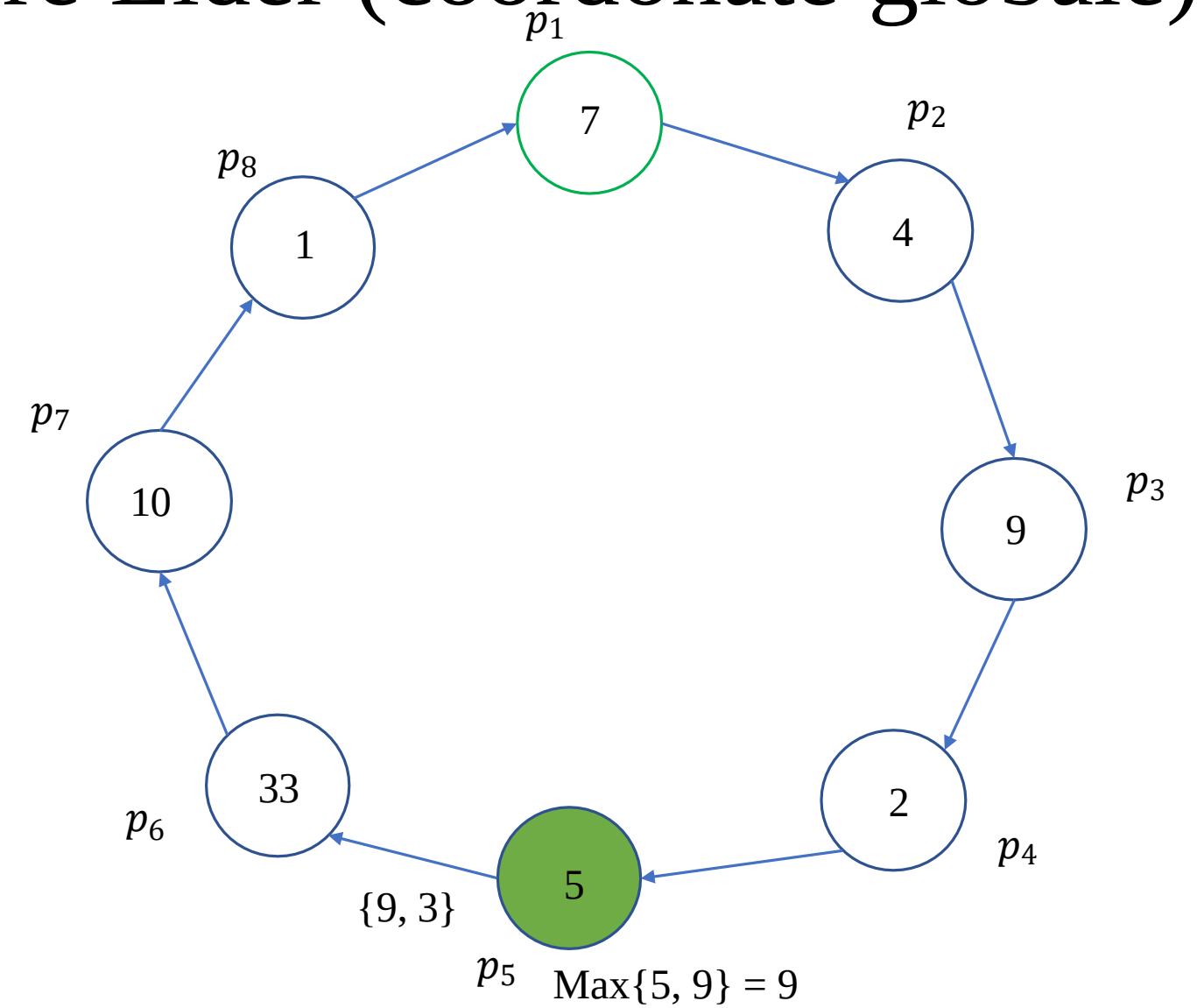
Alegere Lider (coordonate globale)



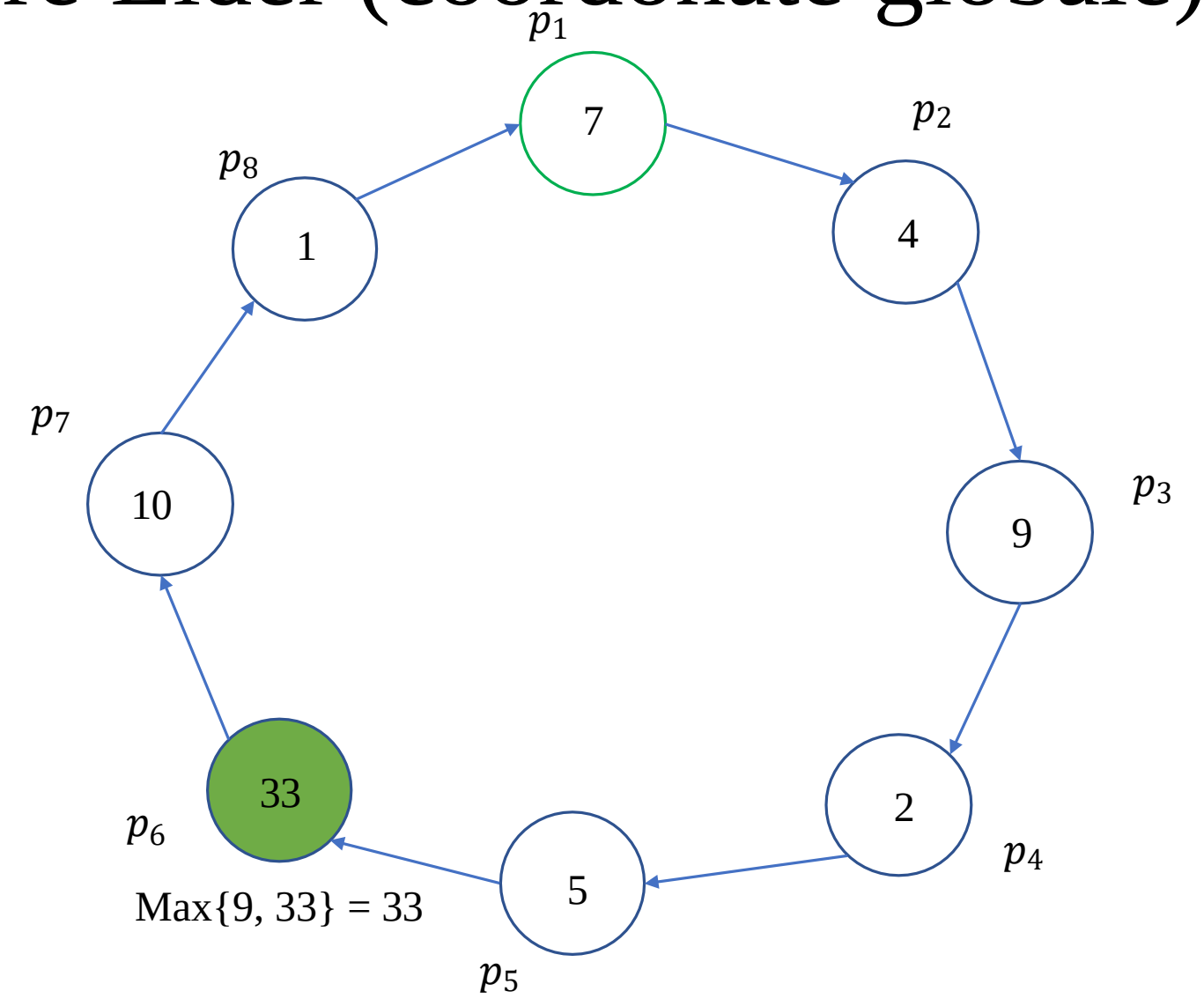
Alegere Lider (coordonate globale)



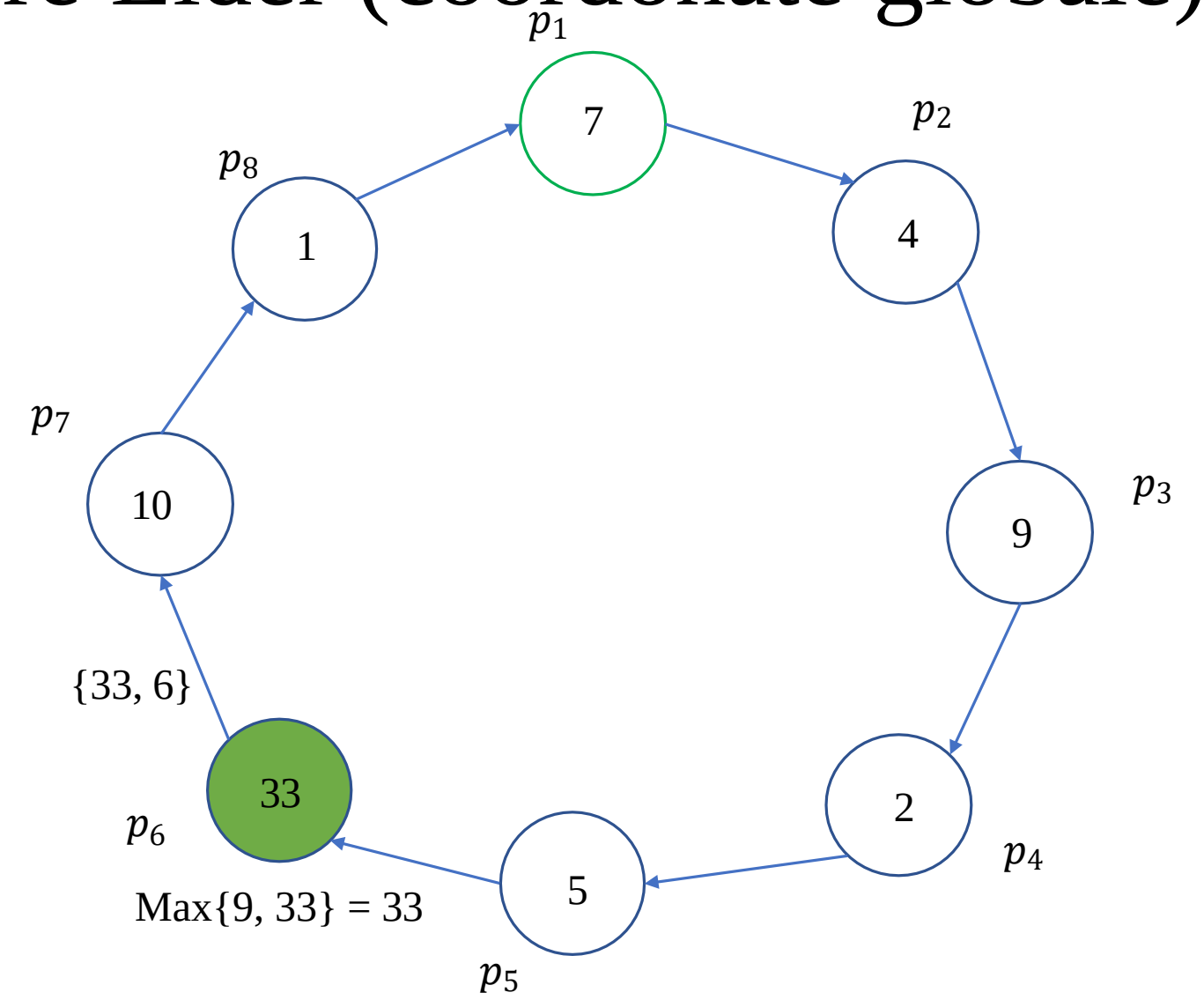
Alegere Lider (coordonate globale)



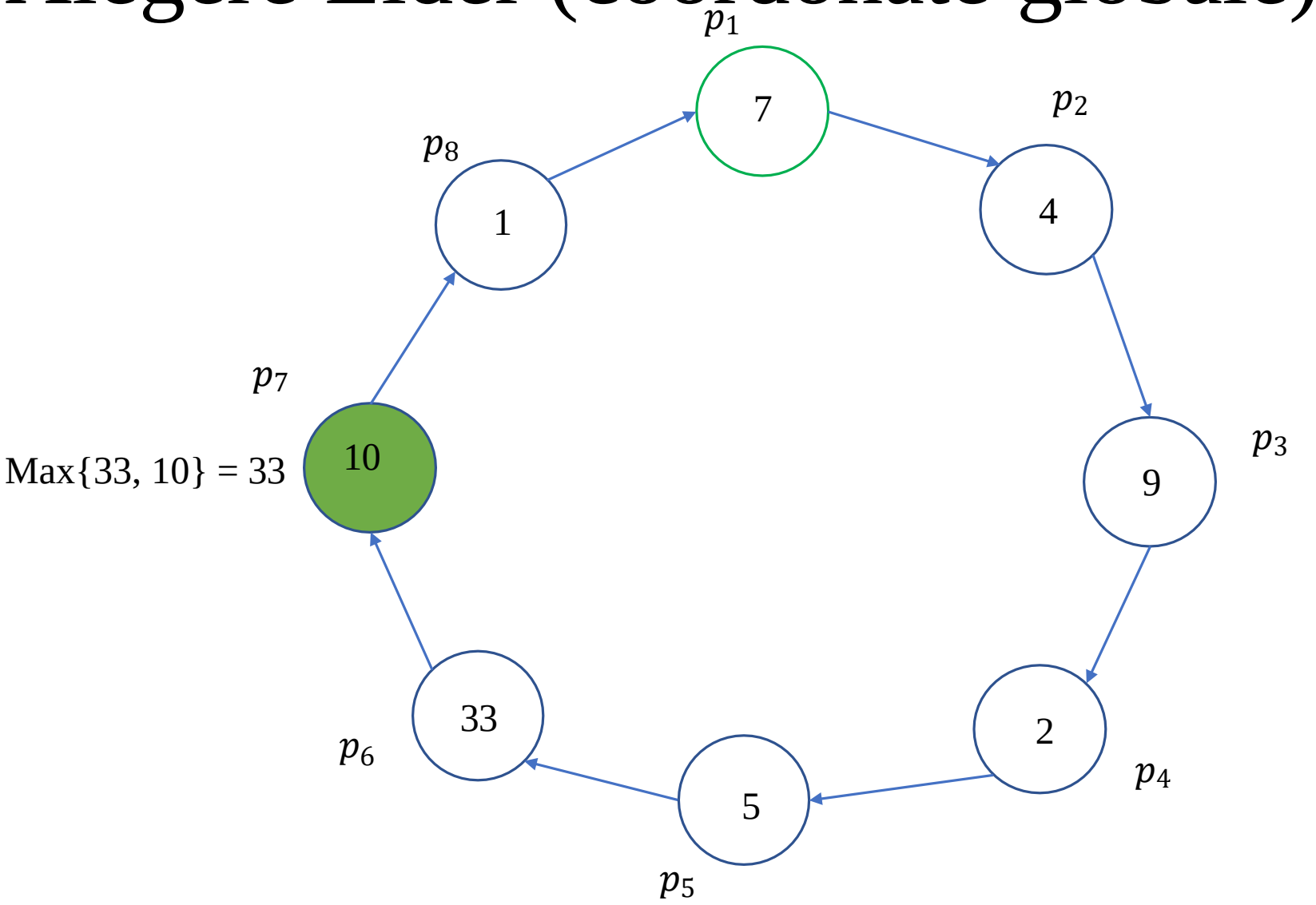
Alegere Lider (coordonate globale)



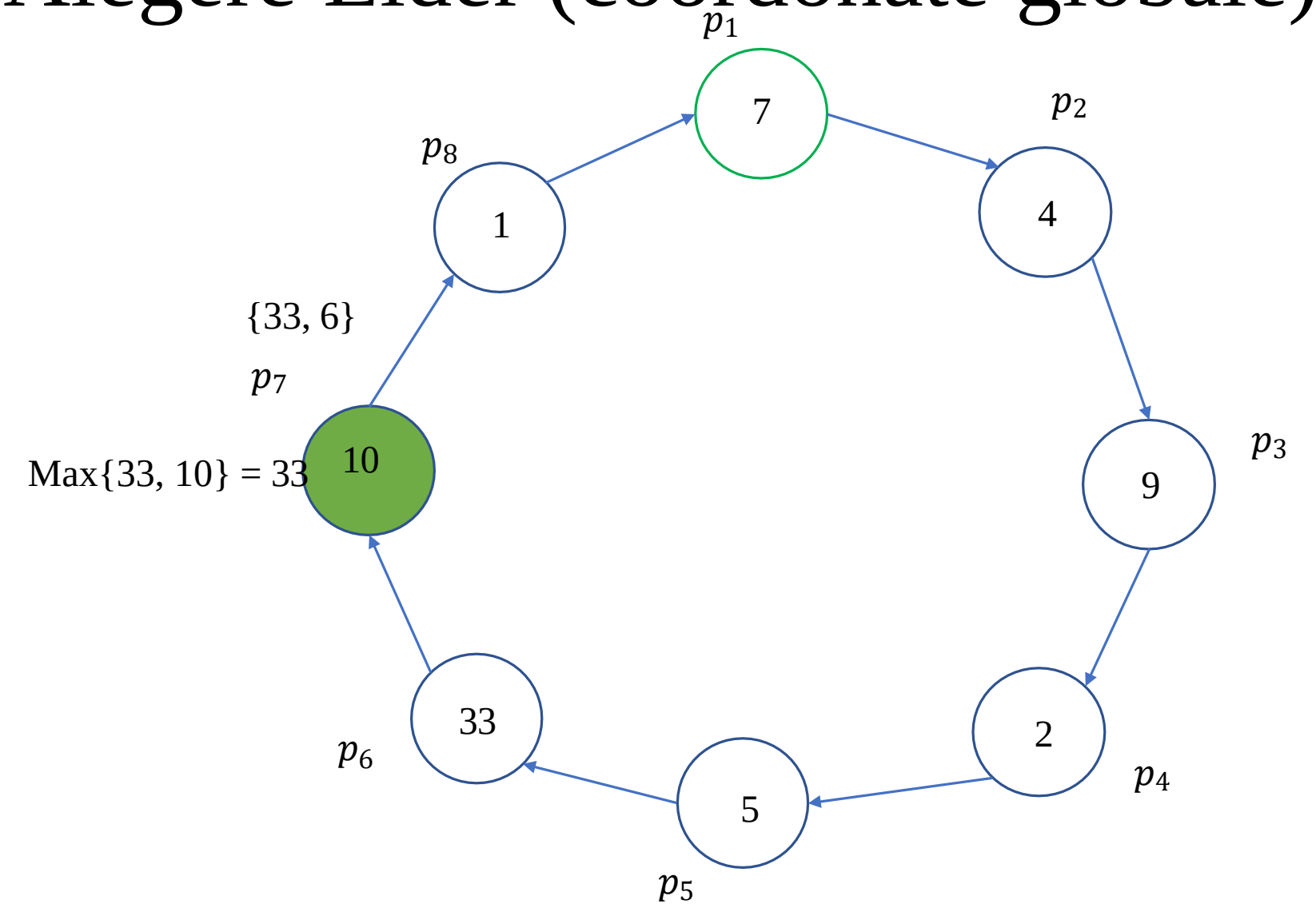
Alegere Lider (coordonate globale)



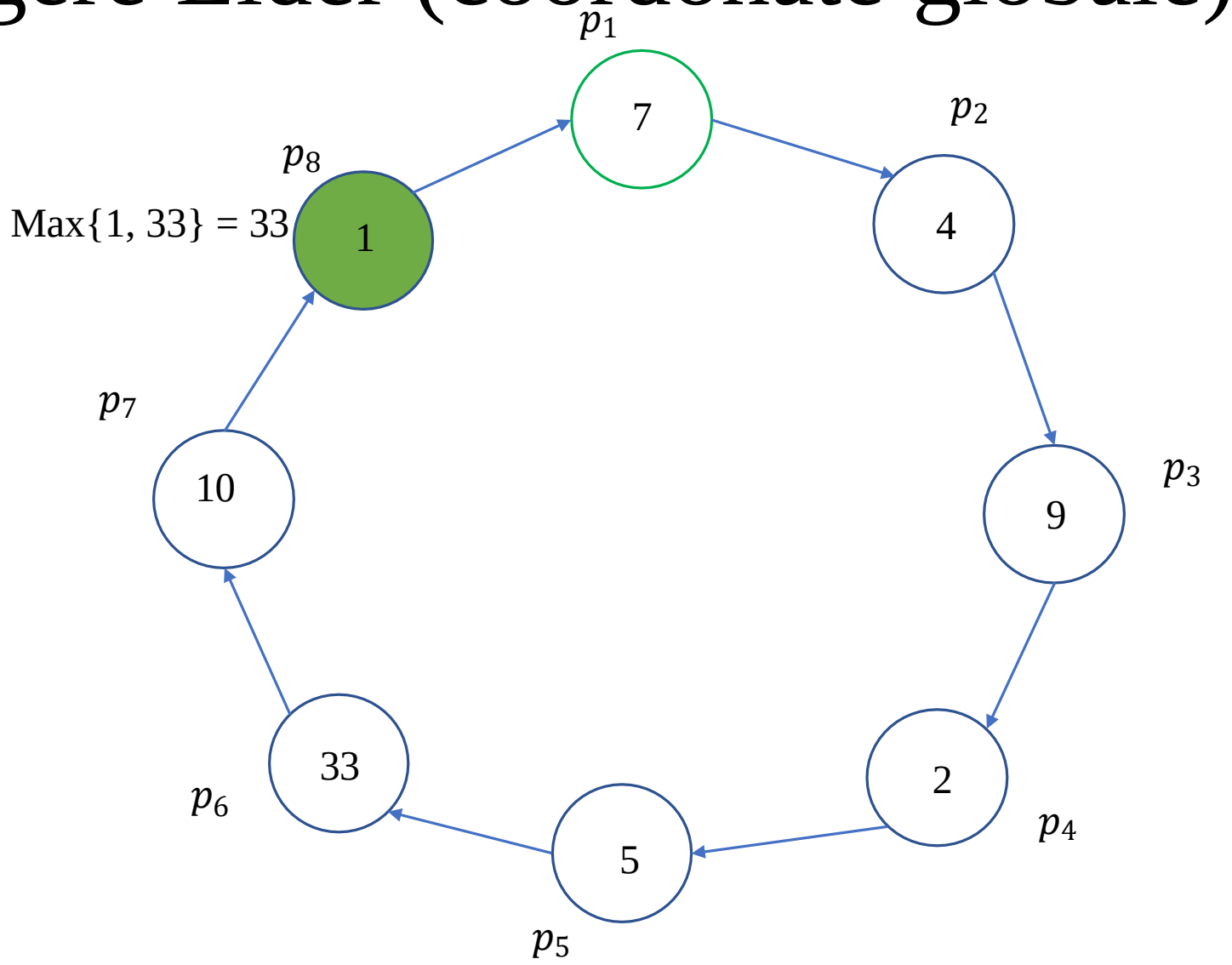
Alegere Lider (coordonate globale)



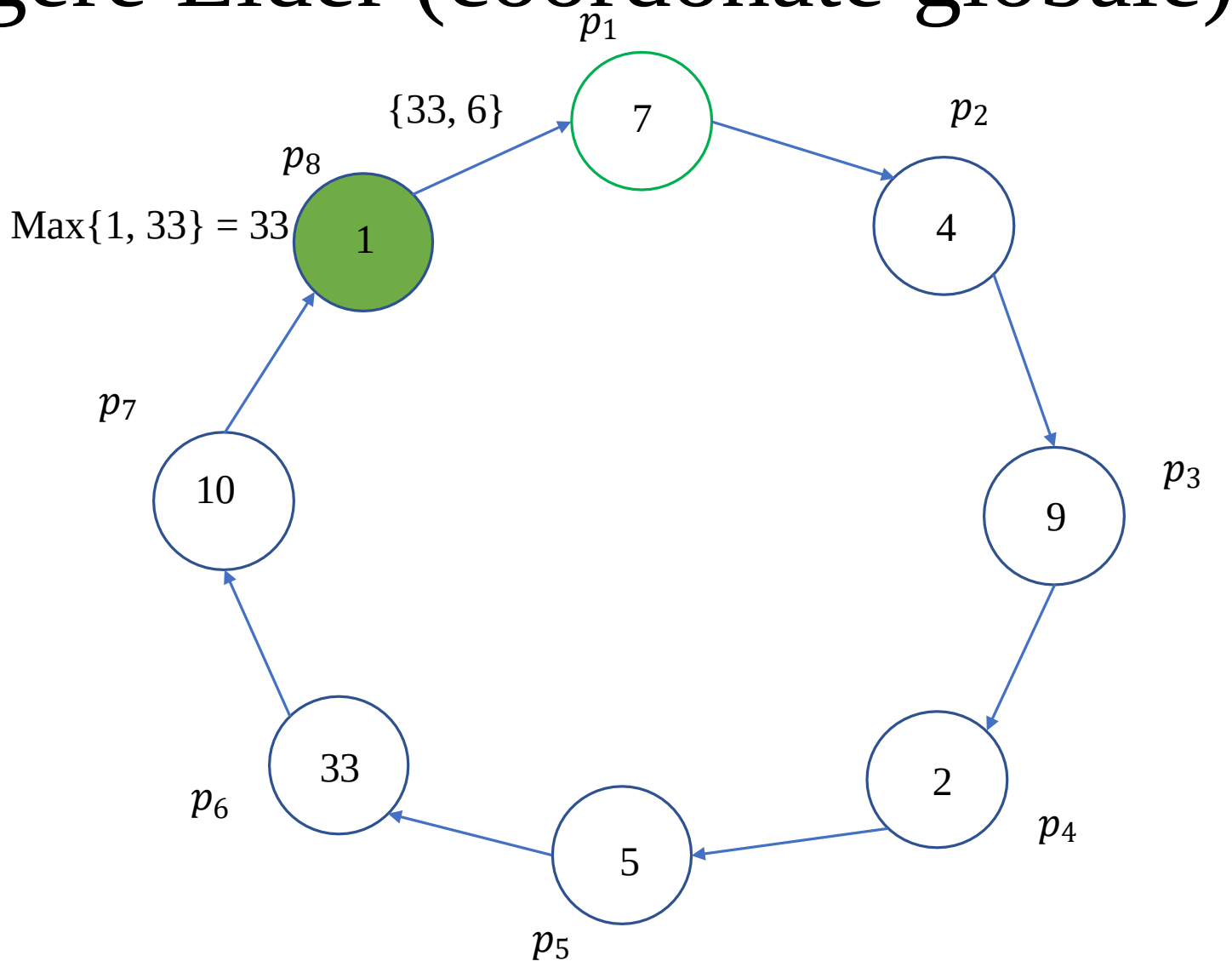
Alegere Lider (coordonate globale)



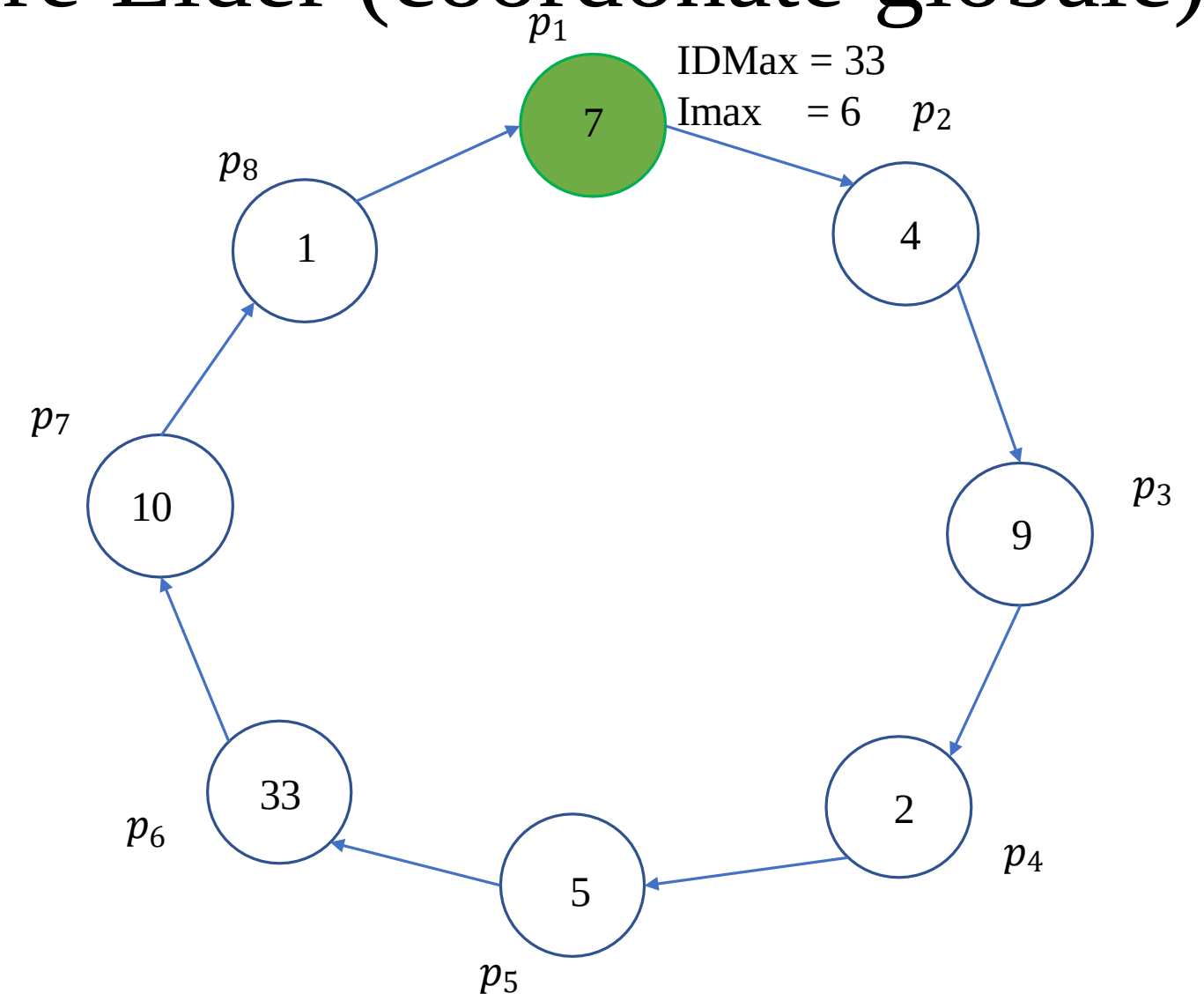
Alegere Lider (coordonate globale)



Alegere Lider (coordonate globale)



Alegere Lider (coordonate globale)



Alegere Lider (coordonate globale)

M_i: - int n (număr noduri)
- int i (index)
- int id (token)
- int id_aux, id_max (memorează id maxim)

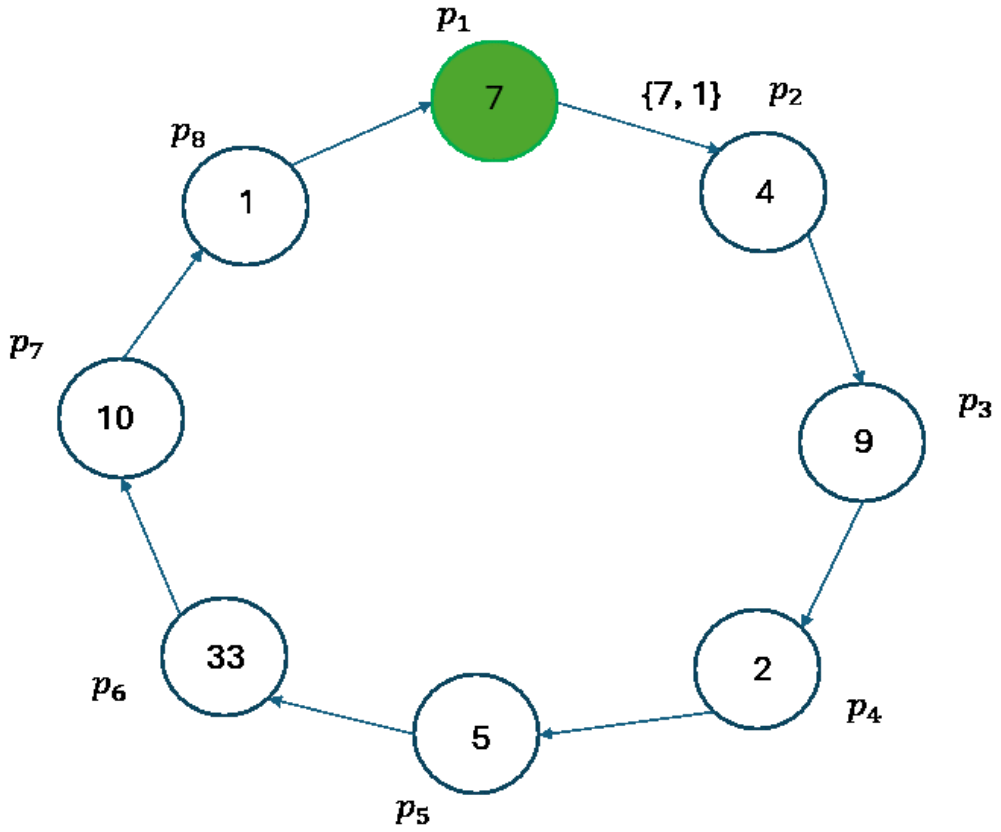
% Faza I: Max ID
…
% Faza II: Difuzare Max ID (Broadcast)

Functie transformare nod i $f_i()$:

1. **If** (i == 1)**:**
    1. send({idmax, imax}, 2);

    Else If (i == n-1)**:**
    1. recv({idmax, imax}, i - 1 mod n);

    Else
    1. recv({idmax, imax}, i - 1 mod n);
    2. send({idmax, imax}, i + 1 mod n);

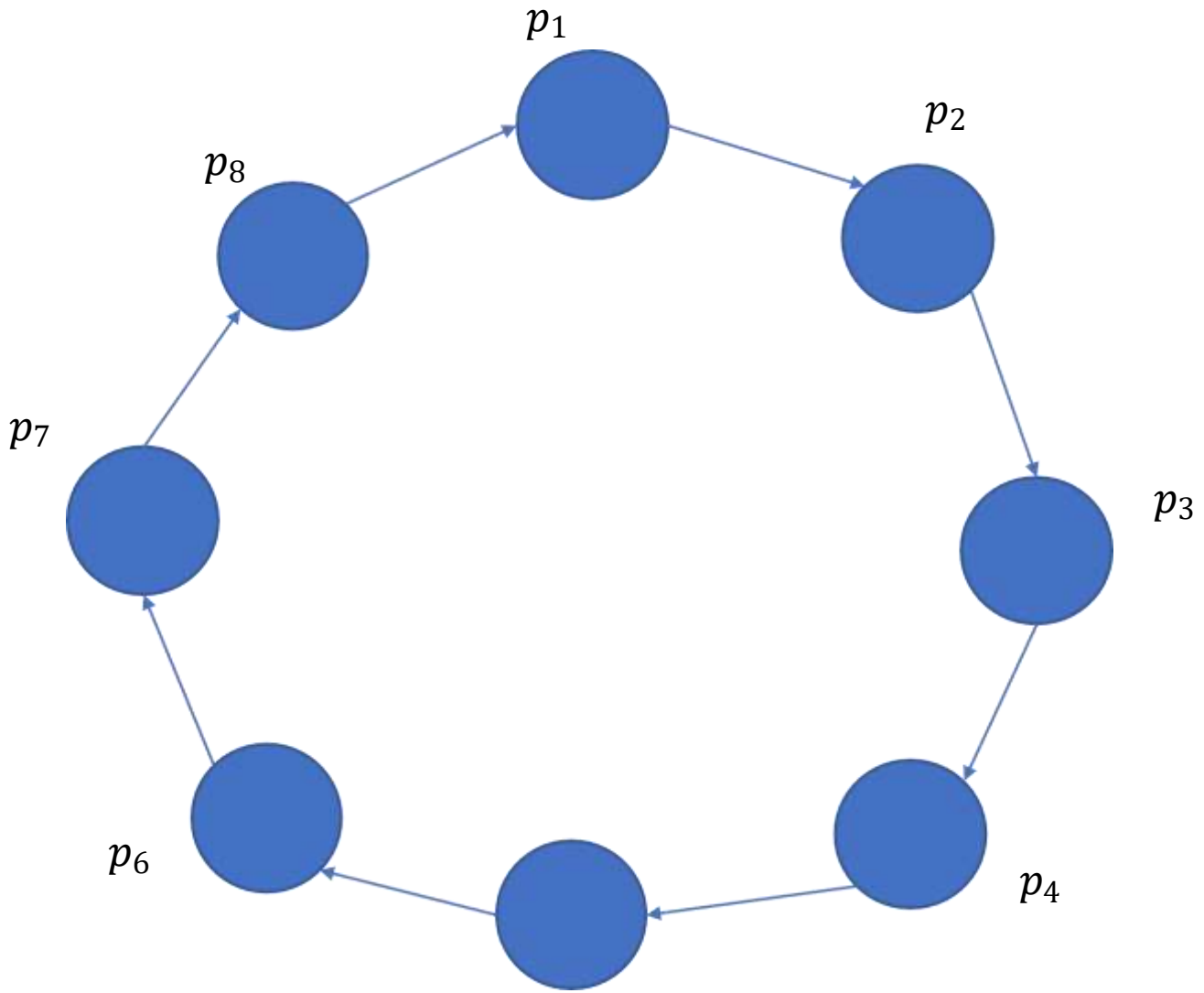


Alegere Lider (AL)

Ipoteze: P_i cunoaște

1. Topologie inel unidirecțional
2. ~~Poziția proprie relativă în topologie (index)~~
3. ~~Numărul de noduri *n*~~

- Ipoteze mai realiste (rețele peer-to-peer)
- Topologia și numărul de noduri sunt statice
- Funcția **AlegereLiderInel()** implică *indexul local* și *numărul de noduri*!

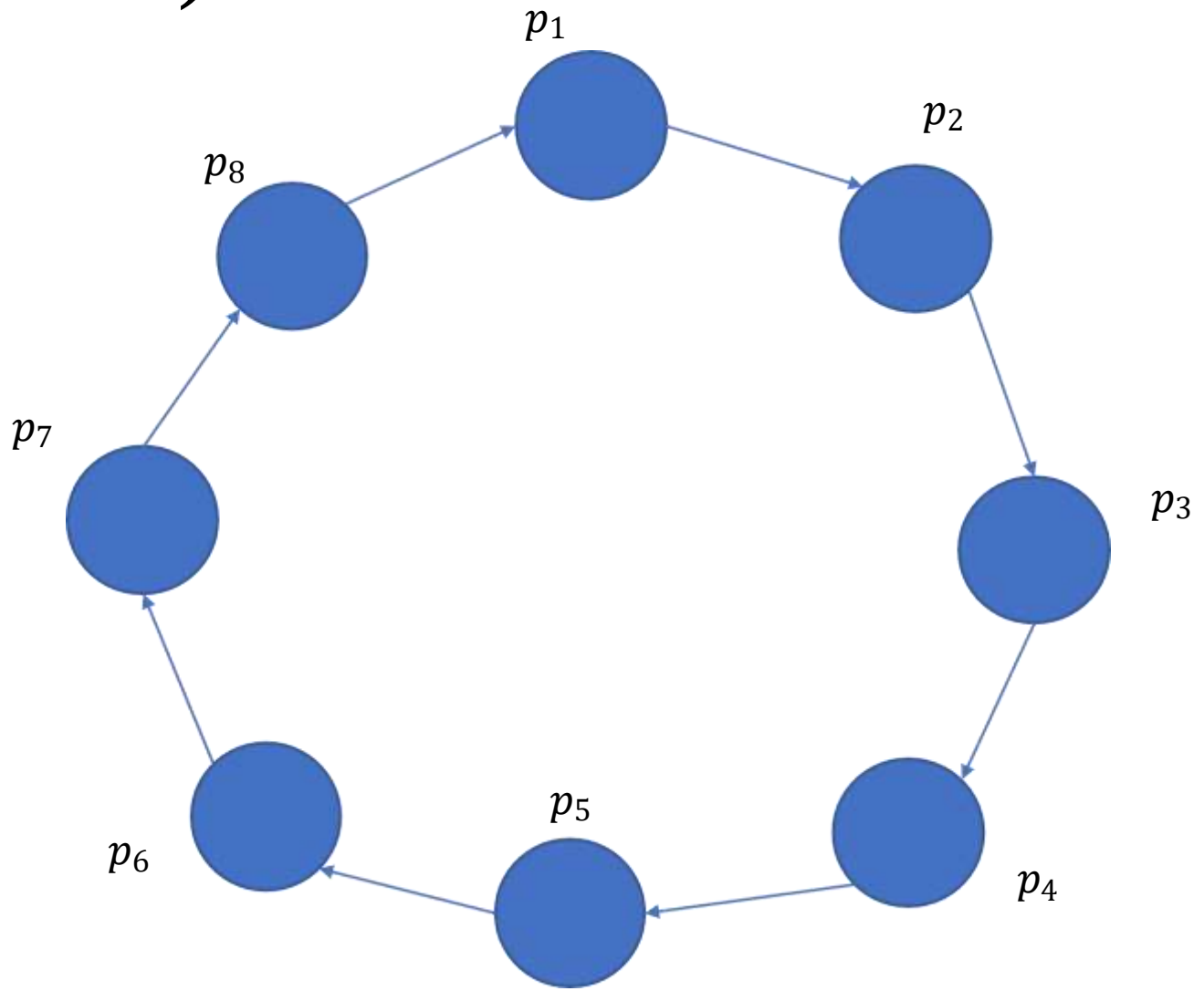


Algoritmul Flooding (LCR)

Algoritm **Flooding**():

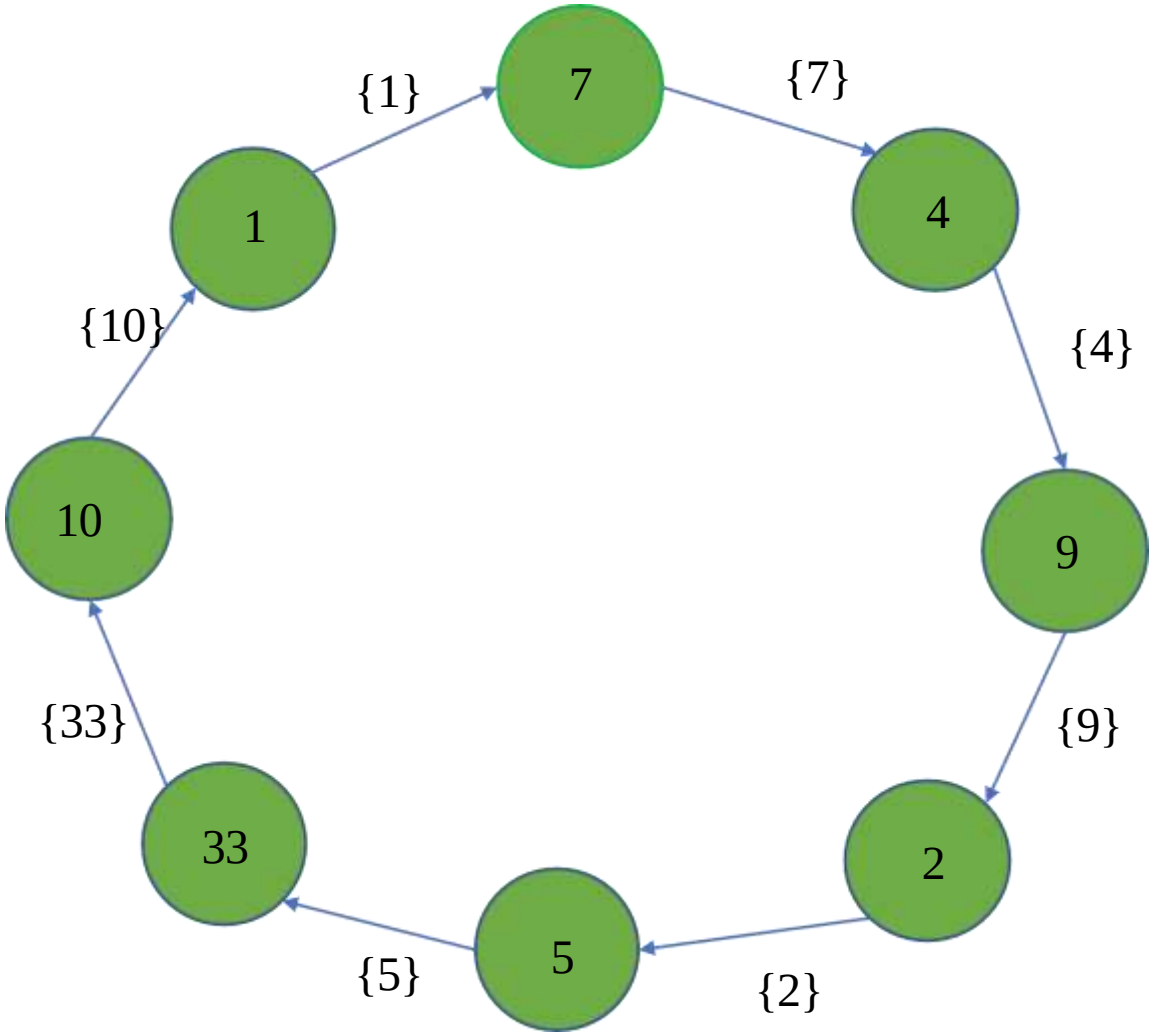
M_i: - int id (token local)
- int send_id, recv_id (var auxiliare), inițial id
- status ∈ {lider, non-lider}, inițial non-lider

Funcție transformare nod i ():
1. send(send_id, dreapta);
2. recv(recv_id, stânga);
3. **If** (recv_id > id):
    1. send_id := recv_id;
4. **ElseIf** (recv_id == id): status = leader;
5. **ElseIf** (recv_id < id): send_id := NULL;



Gerard Le Lann. *Distributed systems – towarda formal approach*. In Bruce Gilchrist, editor, Information Processing 77, Proceedings of IFIP Congress, 155-160, 1977.

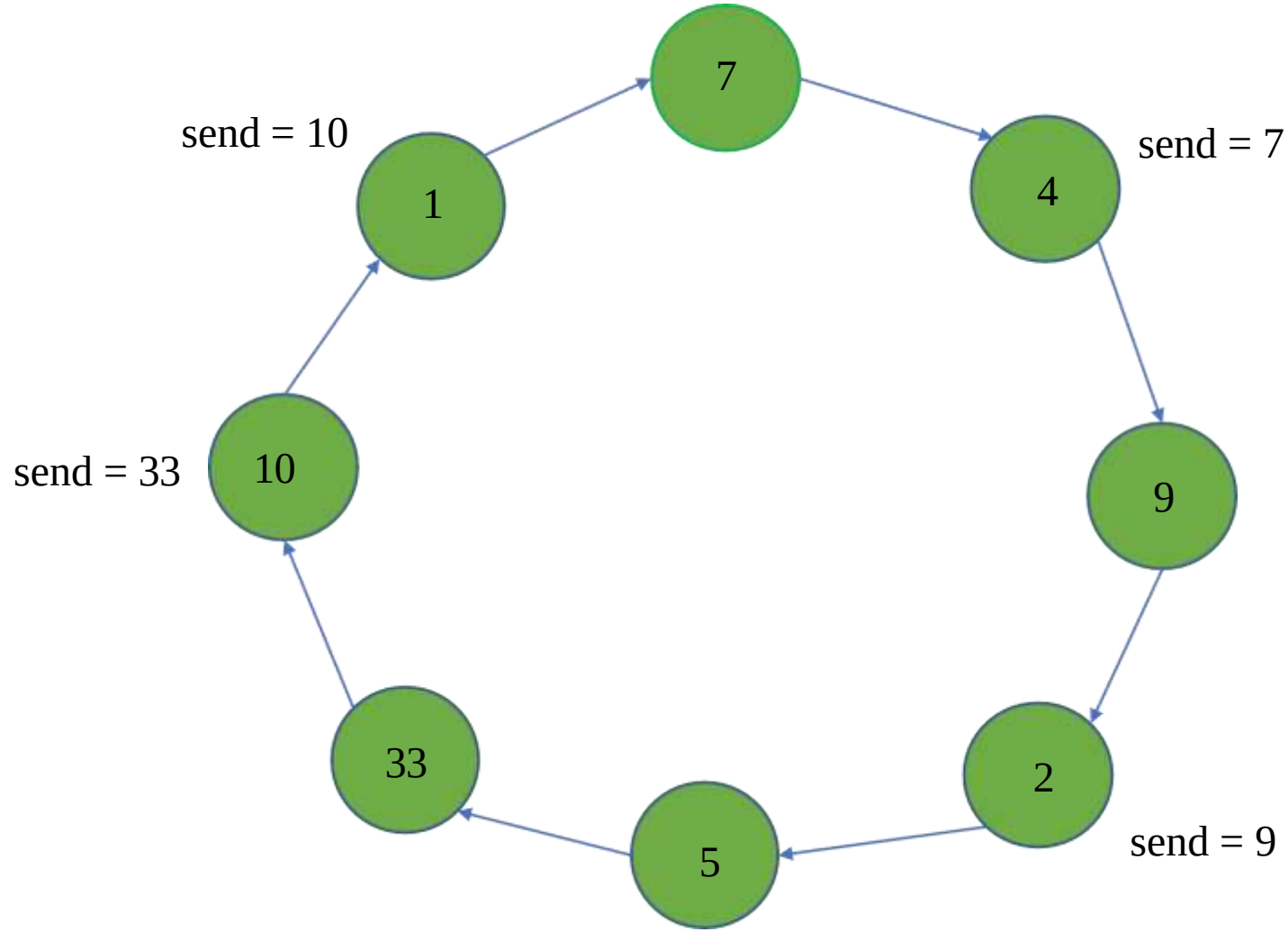
Algoritmul Flooding (LCR)



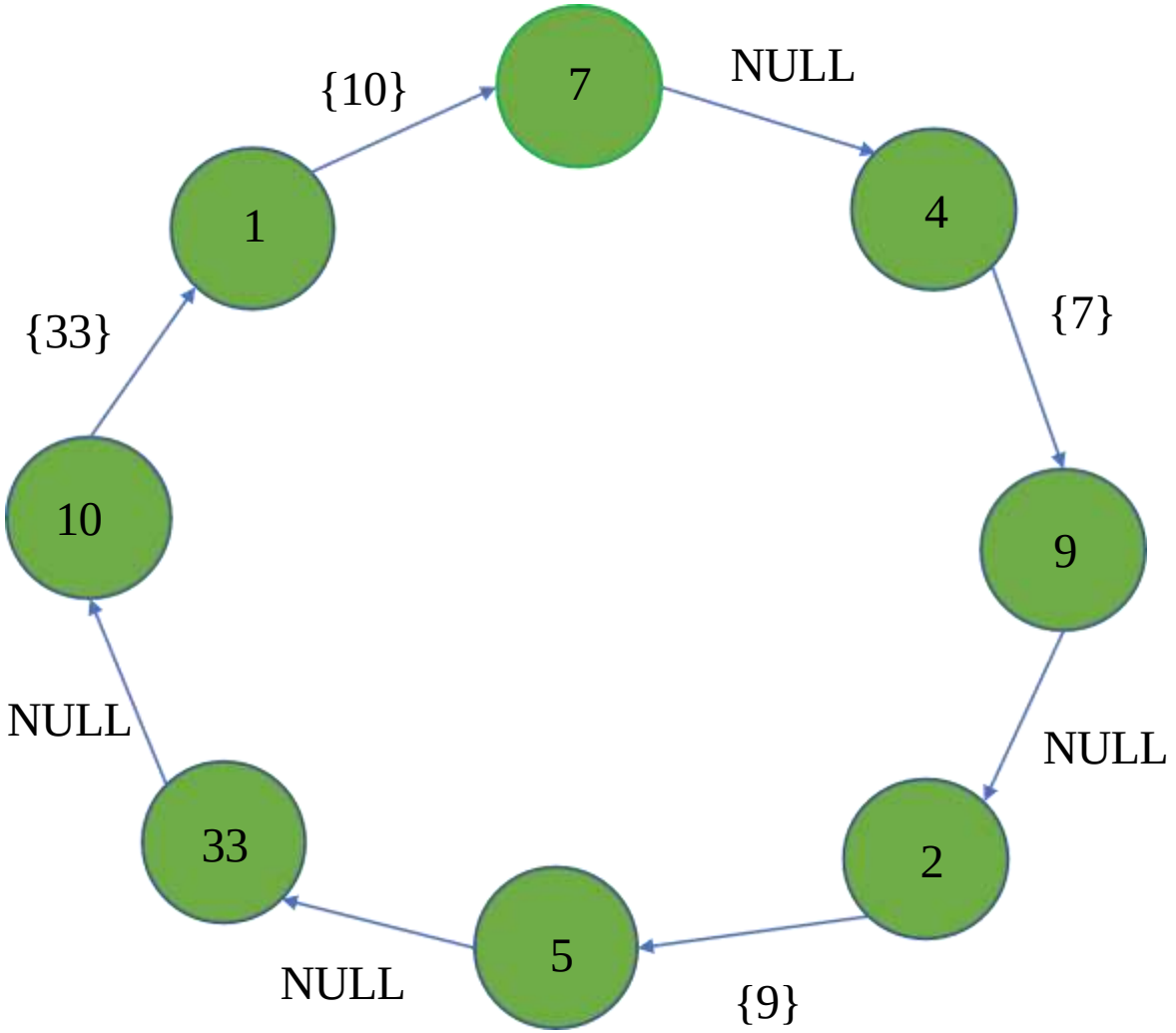
Iterația 1 $x(1)$

Algoritmul Flooding (LCR)

Iterația 1 $x(1)$

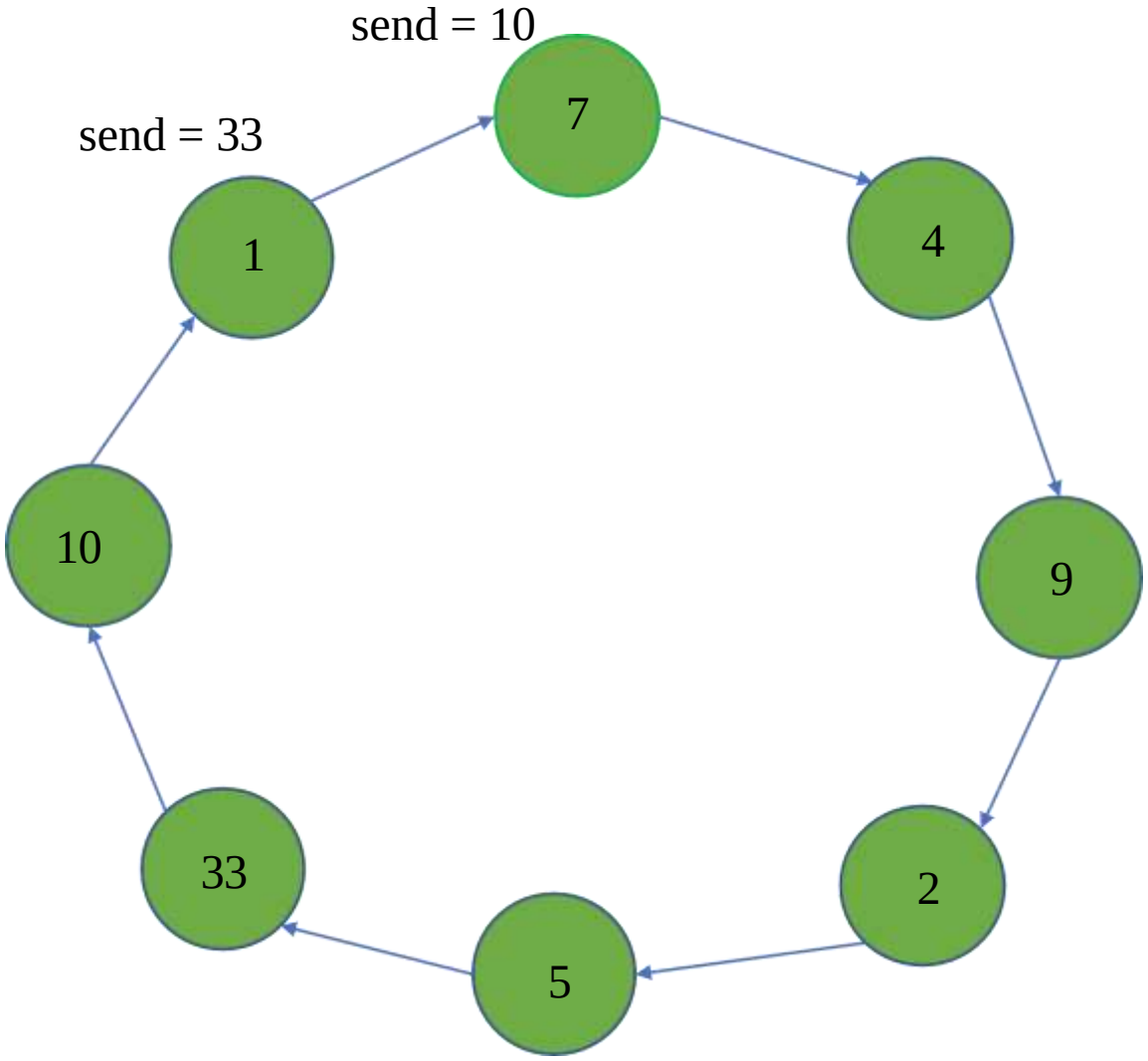


Algoritmul Flooding (LCR)



Iterația 2 $x(2)$

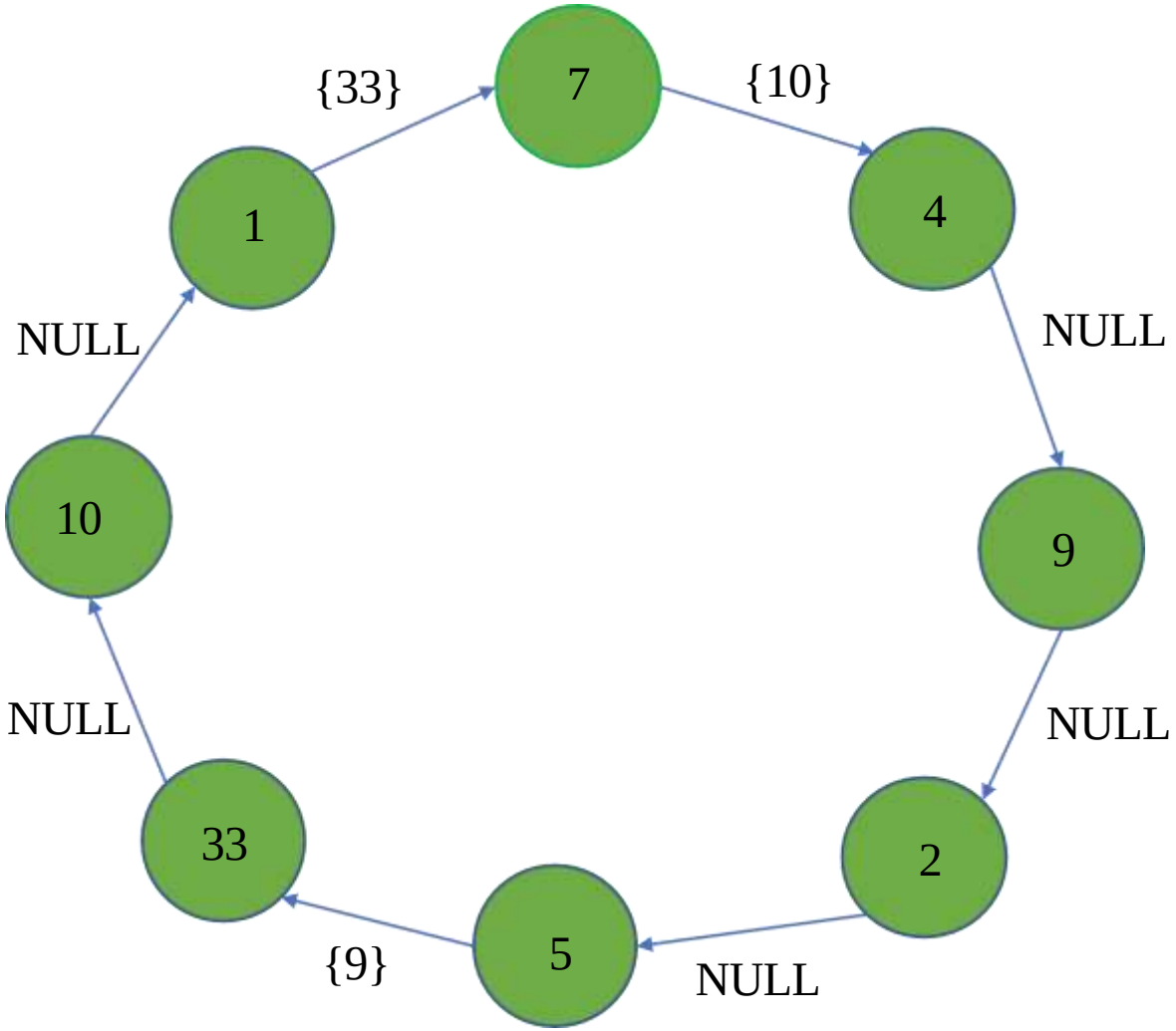
Algoritmul Flooding (LCR)



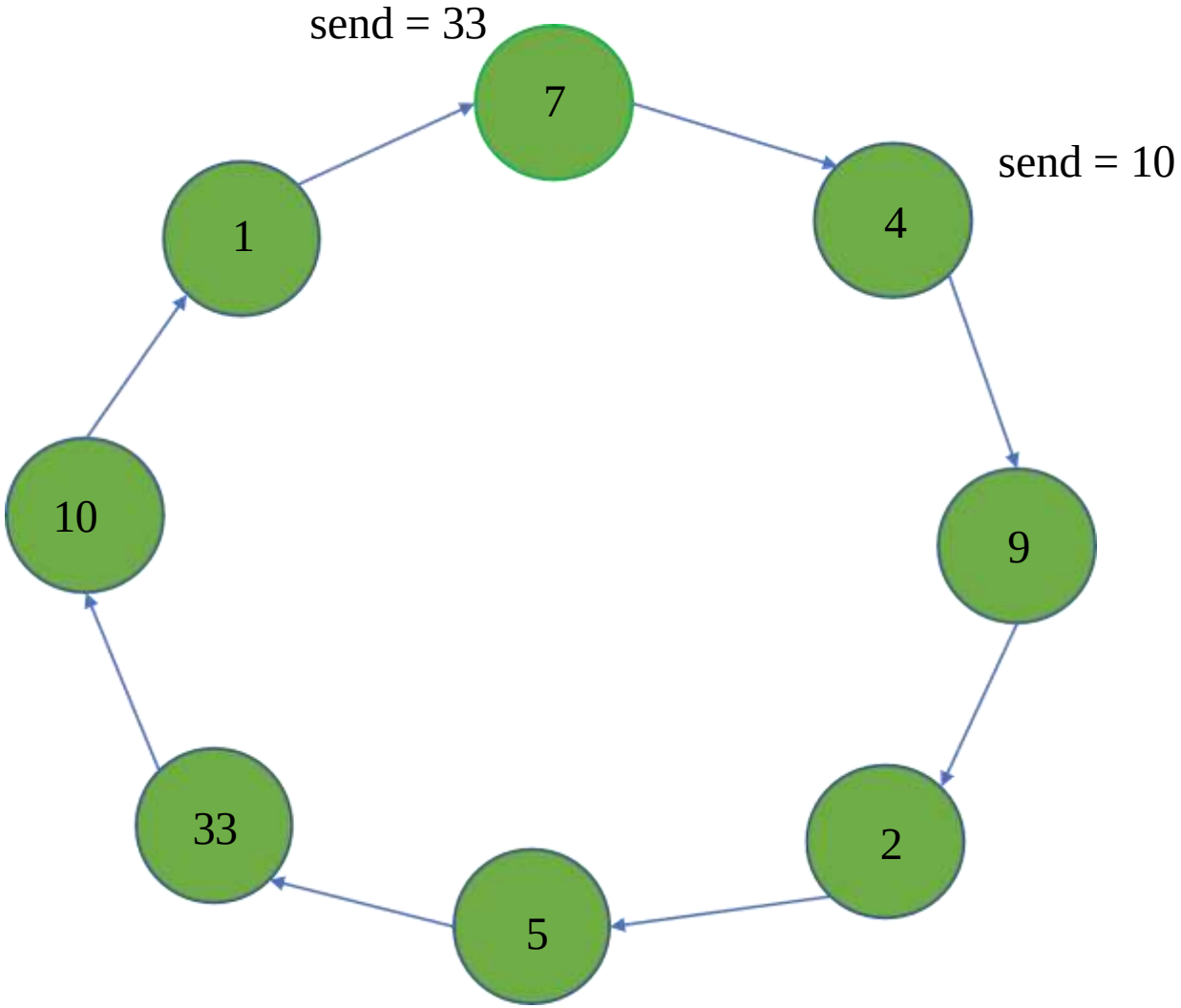
Iterația 2 $x(2)$

Algoritmul Flooding (LCR)

Iterația 3 $x(3)$



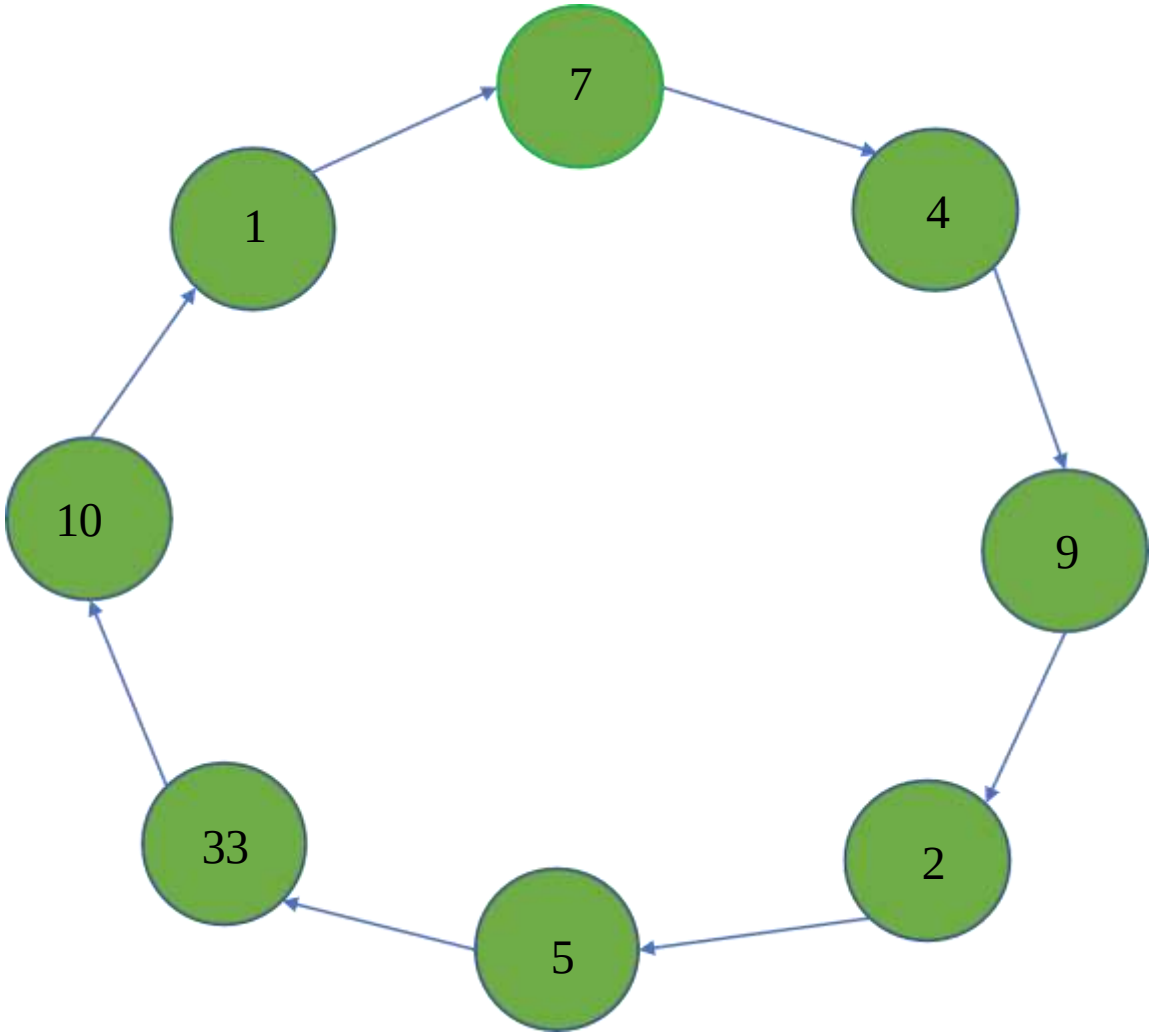
Algoritmul Flooding (LCR)



Iterația 3 $x(3)$

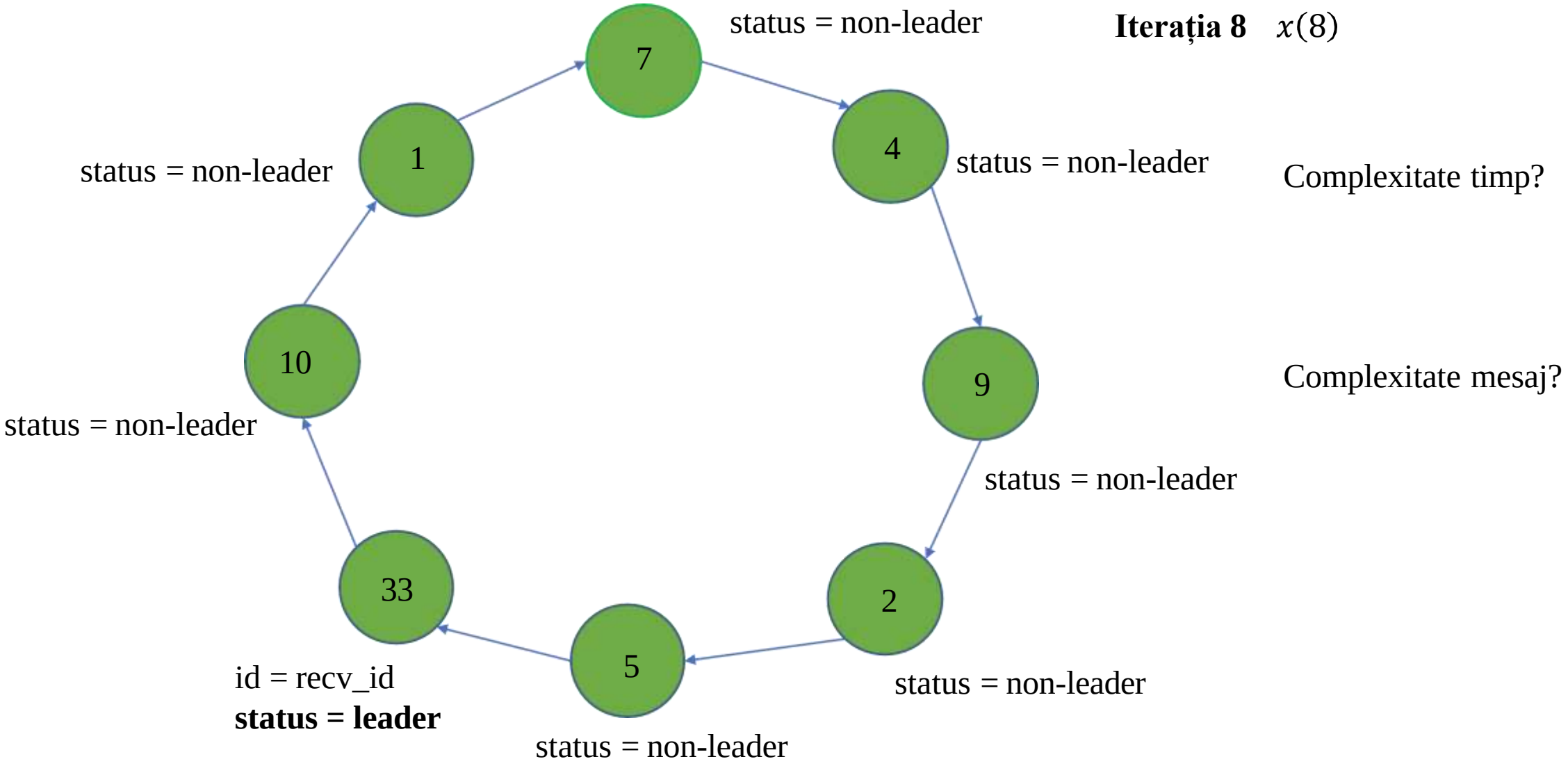
Algoritmul Flooding (LCR)

Iterația 7 $x(7)$

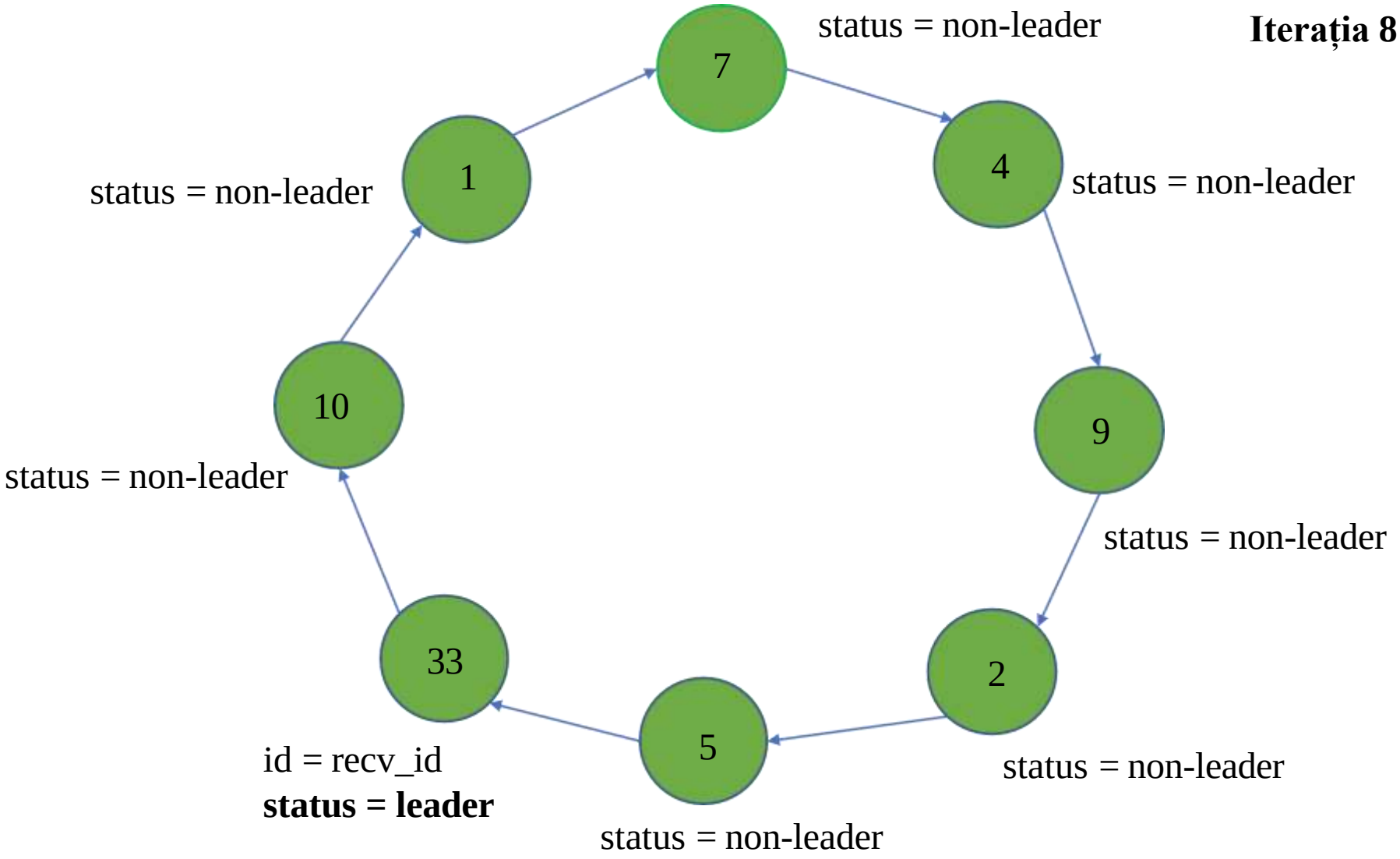


send = 33

Algoritmul Flooding (LCR)



Algoritmul Flooding (LCR)



Iterația 8 $x(8)$

- In varianta curentă nu avem criteriu de oprire (nodurile execută iterația permanent)

- **Soluție**: Pentru oprire liderul poate difuza un mesaj de raport, prin care semnalează încheierea competiției

Algoritmul Flooding (LCR)

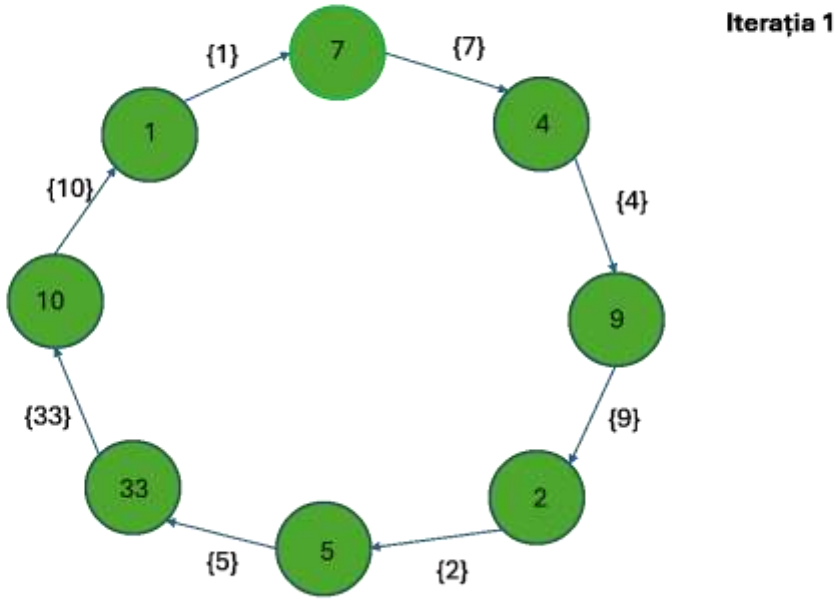
Algoritm **FloodingLCR**():

M_i: - int id (token id local)
- int send_id, recv_id (var auxiliare), inițial id
- status ∈ {lider, non-lider}, inițial non-lider

Funcție transformare nod i ():
1. send(send_id, dreapta);
2. recv(recv_id, stânga);
3. **If** (recv_id > id):
    1. send_id := recv_id;
4. **ElseIf** (recv_id == id): status = leader;
5. **ElseIf** (recv_id < id): send_id := NULL;

Teorema [Lynch]. Algoritmul LCR rezolvă problema alegerii liderului.

Complexitate. Complexitatea timp este n iterații până la anunțarea unui lider, iar complexitatea mesaj este $O(n^2)$**.**



Gerard Le Lann. *Distributed systems – towarda formal approach*. In Bruce Gilchrist, editor, Information Processing 77, Proceedings of IFIP Congress, 155-160, 1977.
Nancy A. Lynch. *Distributed Algorithms*. Morgan Kaufmann Publishers, 1996.

References

1 Seif Haridi, https://canvas.instructure.com/courses/902299/modules

2 A.S. Tanenbaum, M.V. Steen, *DISTRIBUTED SYSTEMS: Principles and Paradigms*, Pearson Prentice Hall, Second Edition, 2007.

3 A. D. Kshemkalyani and M. Singhal, *Distributed Computing: Principles, Algorithms, and Systems*, Cambridge University Press, 2008.

4 G. Tel, *Introduction to Distributed Algorithms*, Cambridge University Press, 2000.

5 M. Raynal, *Distributed Algorithms for Message-Passing Systems*, Springer-Verlag, 2013.